



Міністерство освіти і науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Кафедра АСОІУ

Спеціальність 126 Інформаційні системи та технології

КУРСОВА РОБОТА

з дисципліни

“Додаткові розділи дослідження операцій”

на тему

“Дискретна задача з квадратичною цільовою функцією”

Виконала

студентка групи ІС-81

Власенко В. В.

№ зал. кн. ІС-8108

(підпис)

(оцінка)

Прийняли

Доц. каф. АСОІУ, к.т.н.

ЖДАНОВА О.Г.

(підпис)

Доц. каф. АСОІУ, к.т.н.

ПОПЕНКО В.Д.

(підпис)

Доц. каф. АСОІУ, к.т.н.

СПЕРКАЧ М.О.

(підпис)

*Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»*

(назва вищого навчального закладу)

Кафедра

АСОІУ

Дисципліна

Дослідження операцій

Спеціальність

126 Інформаційні системи та технології

Курс

3

Група

ІС-81

Семестр

6

**ЗАВДАННЯ
НА КУРСОВУ РОБОТУ СТУДЕНТЦІ**

Власенко Вікторії Віталіївні

1. Тема роботи *Дискретна задача з квадратичною цільовою функцією*

2. Термін здачі студентом закінченої роботи *30 травня 2021 року*

3. Завдання курсової роботи

- 1) Побудувати математичну модель задачі.
- 2) Виконати аналіз можливих методів розв'язання.
- 3) Розробити жадібний алгоритм розв'язання задачі.
- 4) Розробити бджолиний алгоритм розв'язання задачі.
- 5) Виконати програмну реалізацію бджолиного алгоритму.
- 6) Розробити генетичний алгоритм розв'язання задачі.
- 7) Виконати програмну реалізацію генетичного алгоритму.
- 8) Програмно реалізувати генератор індивідуальних задач.
- 9) Експериментально дослідити розроблені алгоритми, виконати їх порівняльний аналіз.
- 10) Визначити складність розроблених алгоритмів (теоретичну та експериментальну).

4. Зміст розрахунково-пояснювальної записки

Змістовні постановки задач. Математична постановка задачі.

Аналіз можливих методів розв'язання. Опис розроблених алгоритмів.

Опис програмного продукту. Результати експериментального дослідження алгоритмів.

Оцінка складності розроблених алгоритмів. Висновки

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

Відсутній.

6. Дата видачі завдання

10 лютого 2021 року

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів виконання курсової роботи	Строк виконання етапів роботи	Примітка
1	<i>Опис постановки задачі</i>	06.03	
2	<i>Пошук літературних джерел за темою курсової роботи. Порівняльний аналіз існуючих методів розв'язання задачі</i>	15.03	
3	<i>Вивчення теоретичних положень жадібних, бджолиного та генетичного алгоритмів</i>	31.03	
4	<i>Розробка жадібного, бджолиного та генетичного алгоритмів</i>	10.04	
5	<i>Розв'язання задач "вручну"</i>	15.04	
6	<i>Консультація в керівника – погодження інтерфейсу програми</i>	15.04	
7	<i>Програмна реалізація алгоритмів та генератору індивідуальних задач. Відлагодження програм</i>	10.05	
8	<i>Експериментальне дослідження розроблених алгоритмів, їх порівняльний аналіз. Оцінка складності алгоритмів розв'язання задачі</i>	15.05	
9	<i>Оформлення пояснювальної записки</i>	20.05	
10	<i>Захист курсової роботи</i>	31.05	

Студентка

Власенко В. В.

(підпис)

(прізвище, ініціали)

Керівник

Жданова О.Г.

(підпис)

(прізвище, ініціали)

«20» лютого 2021 року

ЗМІСТ

1. ПОСТАНОВКА ЗАДАЧІ	7
1.1. Математична постановка задачі	7
1.2. Постановка індивідуальних задач	7
1.2.1. Задача на максимум	7
1.2.2. Задача на мінімум	11
2. ОПИС МЕТОДІВ РОЗВ'ЯЗАННЯ ЗАДАЧІ	15
2.1 Наявні методи розв'язання	15
2.2 Розробка жадібного алгоритму розв'язання задачі	17
2.3 Опис бджолиного алгоритму із застосуванням локального пошуку	18
2.3.1 Розробка алгоритму розв'язання задачі	19
2.3.2 Оцінка трудомісткості бджолиного алгоритму із застосуванням локального пошуку	21
2.4 Опис Генетичного алгоритму	21
2.4.1 Розробка Генетичного алгоритму розв'язання задачі	23
2.4.2 Оцінка трудомісткості алгоритму Генетичного алгоритму	30
2.5 Приклади застосування розроблених алгоритмів	30
2.5.1 Приклад розв'язання жадібним алгоритмом	30
2.5.2 Приклад розв'язання задачі на максимум бджолиним алгоритмом із застосуванням локального пошуку	32
2.5.3 Приклад розв'язання задачі Генетичним алгоритмом	40
2.6 Джерела	53
3. ПЛАНУВАННЯ ЕКСПЕРИМЕНТІВ	54
3.1 Класифікація індивідуальних задач	54
3.2 Параметри алгоритмів	54
3.3 План експериментів	55
4. ОПИС ПРОГРАМНОГО ПРОДУКТУ	58
4.1 Опис функціональної моделі	58
4.2 Рішення з інформаційного забезпечення	59
4.2.1 Вхідні дані	59
4.2.2 Вихідні дані	61
4.2.3 Опис інформаційного забезпечення	61
4.3 Рішення з програмного забезпечення	61

4.3.1	Опис обраних засобів розробки.....	61
4.3.2	Архітектура програмного забезпечення	61
4.4	Керівництво користувача	63
4.4.1	Інструкція користувача	63
4.4.2	Методика випробувань.....	64
5.	ДОСЛІДЖЕННЯ РОЗРОБЛЕНИХ АЛГОРИТМІВ.....	65
5.1	Дослідження впливу розмірності ІЗ на значення ЦФ	65
5.2	Дослідження впливу кількості ітерацій роботи алгоритмів на значення ЦФ	67
5.3	Дослідження впливу значення константи В на значення ЦФ	70
5.4	Дослідження впливу типів селекції генетичного алгоритму на значення ЦФ	72
5.5	Дослідження впливу наявності та відсутності локальної оптимізації генетичного алгоритму на значення ЦФ.	77

1. ПОСТАНОВКА ЗАДАЧІ

1.1. Математична постановка задачі

Нехай маємо деяку систему параметрів, якими можна керувати.

Змінні:

Дано n змінних, n - парне.

Цільова функція:

$$x_1 x_2 + x_3 x_4 + \dots x_{n-1} x_n \rightarrow extremum$$

Обмеження:

$$\sum_{j=1}^n x_j \leq B, \quad (1)$$

$$a_j \leq x_j \leq b_j, \quad j = 1, 2, \dots, n, \quad (2)$$

$$x_j \in Z, \quad j = 1, 2, \dots, n, \quad (3)$$

$$a_j \leq 0, \quad j = 1, 2, \dots, n, \quad (4)$$

$$b_j > 0, \quad j = 1, 2, \dots, n, \quad (5)$$

$$B < \sum_{j=1}^n b_j. \quad (6)$$

Знайти такі значення x_j , при яких цільова функція досягає свого екстремального значення.

1.2. Постановка індивідуальних задач

1.2.1. Задача на максимум

Змінні:

Дано n змінних, n - парне.

Знайти такі значення x_j , при яких цільова функція досягає свого максимального значення.

Цільова функція:

$$x_1 x_2 + x_3 x_4 + \dots x_{n-1} x_n \rightarrow max \quad (7)$$

Обмеження:

$$\sum_{j=1}^n x_j \leq B,$$

$$a_j \leq x_j \leq b_j, \quad j = 1, 2, \dots, n,$$

$$x_j \in Z, \quad j = 1, 2, \dots, n,$$

$$a_j \leq 0, \quad j = 1, 2, \dots, n,$$

$$b_j > 0, \quad j = 1, 2, \dots, n,$$

$$B < \sum_{j=1}^n b_j.$$

Задача №1

Значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ для задачі на максимум наведені у таблиці 1.1.

Таблиця 1.1 - Значення вхідних параметрів (задача на максимум)

Назва масиву параметрів	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
a_j	-14	12	10	0	-24	3	13	7	-13	2
b_j	10	19	22	11	1	10	13	19	1	20

$$B = 115.$$

У таблиці 1.2 наведений допустимий розв'язок №1 для задачі на максимум.

Таблиця 1.2 - Допустимий розв'язок №1 (задача на максимум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	-1	13	21	2	-19	10	13	19	-4	3

$$\sum_{j=1}^n x_j = 57 \leq B - \text{виконується обмеження (1).}$$

$$\sum_{j=1}^n b_j = 10 + 19 + 22 + 11 + 1 + 10 + 13 + 19 + 1 + 20 = 126 > B - \text{виконується обмеження (6).}$$

Розв'язок №1:

$$z = (-1) * 13 + 21 * 2 + (-19) * 10 + 13 * 19 + (-4) * 3 = 74.$$

У таблиці 1.3 наведений допустимий розв'язок №2 для задачі на максимум.

Таблиця 1.3 - Допустимий розв'язок №2 (задача на максимум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	-4	7	21	5	-20	7	13	10	-5	17

$$\sum_{j=1}^n x_j = 51 \leq B - \text{виконується обмеження (1).}$$

$$\sum_{j=1}^n b_j = 10 + 19 + 22 + 11 + 1 + 10 + 13 + 19 + 1 + 20 = 126 > B - \text{виконується обмеження (6).}$$

Розв'язок №2:

$$z = (-4) * 7 + 21 * 5 + (-20) * 7 + 13 * 10 + (-5) * 17 = -18.$$

Висновок:

$$\max(74, -18) = 74.$$

Допустимий розв'язок №1 кращий від допустимого розв'язку №2.

Задача №2

Значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ для задачі на максимум наведені у таблиці 1.4.

Таблиця 1.4 - Значення вхідних параметрів (задача на максимум)

Назва масиву параметрів	Порядковий елемент $j = 1, 2, \dots, n, n = 12$											
	1	2	3	4	5	6	7	8	9	10	11	12
a_j	0	-1	5	-10	-20	17	2	-5	10	-21	0	5
b_j	4	5	6	7	8	21	20	9	18	10	19	11

$$B = 75.$$

У таблиці 1.5 наведений допустимий розв'язок №1 для задачі на максимум.

Таблиця 1.5 - Допустимий розв'язок №1 (задача на максимум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 12$											
	1	2	3	4	5	6	7	8	9	10	11	12
x_j	3	-9	6	2	-17	20	19	-16	15	-12	18	8

$\sum_{j=1}^n x_j = 43 \leq B$ - виконується обмеження (1).

$\sum_{j=1}^n b_j = 4 + 5 + 6 + 7 + 8 + 21 + 20 + 9 + 18 + 10 + 19 + 11 = 138 > B$ - виконується обмеження (6).

Розв'язок:

$$z = 3 * (-9) + 6 * 2 + (-17) * 20 + 19 * (-16) + 15 * (-12) + 18 * 8 = -695.$$

У таблиці 1.6 наведений допустимий розв'язок №2 для задачі на максимум.

Таблиця 1.6 - Допустимий розв'язок №2 (задача на максимум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 12$											
	1	2	3	4	5	6	7	8	9	10	11	12
x_j	3	4	5	6	7	8	9	8	10	9	11	10

$\sum_{j=1}^n x_j = 90 \leq B$ - виконується обмеження (1).

$\sum_{j=1}^n b_j = 4 + 5 + 6 + 7 + 8 + 21 + 20 + 9 + 18 + 10 + 19 + 11 = 138 > B$ - виконується обмеження (6).

Розв'язок:

$$z = 3 * 4 + 5 * 6 + 7 * 8 + 9 * 8 + 10 * 9 + 11 * 10 = 370.$$

Висновок:

$$\max(-695, 370) = 370.$$

Допустимий розв'язок №2 кращий від допустимого розв'язку №1.

1.2.2. Задача на мінімум

Змінні:

Дано n змінних, n - парне.

Знайти такі значення x_j , при яких цільова функція досягає свого мінімального значення.

Цільова функція:

$$x_1x_2 + x_3x_4 + \dots x_{n-1}x_n \rightarrow \min$$

Обмеження:

$$\sum_{j=1}^n x_j \leq B,$$

$$a_j \leq x_j \leq b_j, \quad j = 1, 2, \dots, n,$$

$$x_j \in Z, \quad j = 1, 2, \dots, n,$$

$$a_j \leq 0, \quad j = 1, 2, \dots, n,$$

$$b_j > 0, \quad j = 1, 2, \dots, n,$$

$$B < \sum_{j=1}^n b_j.$$

Задача №3

Значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ для задачі на мінімум наведені у таблиці 1.7.

Таблиця 1.7 - Значення вхідних параметрів (задача на мінімум)

Назва масиву параметрів	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
a_j	1	-30	15	11	-14	-11	-1	-7	-20	8
b_j	3	2	24	11	30	4	1	13	27	11

$$B = 120.$$

$$\sum_{j=1}^n b_j = 126 > B \text{ (задовольняє обмеження (6)).}$$

У таблиці 1.8 наведений допустимий розв'язок №1 для задачі на мінімум.

Таблиця 1.8 - Допустимий розв'язок №1 (задача на мінімум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	2	-8	23	11	20	-3	-1	-6	12	11

Оскільки $\sum_{j=1}^n x_j = 61 \leq B$ (задовольняє обмеження (1)), то далі знаходимо допустимий розв'язок:

$$z = 2 * (-8) + 23 * 11 + 20 * (-3) - 1 * (-6) + 12 * 11 = 315.$$

Розв'язок №1:

$$z = 315.$$

У таблиці 1.9 наведений допустимий розв'язок №2 для задачі на мінімум.

Таблиця 1.9 - Допустимий розв'язок №2 (задача на мінімум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	2	1	18	11	9	4	1	-6	-3	9

Оскільки $\sum_{j=1}^n x_j = 46 \leq B$ (задовольняє обмеження (1)), то далі знаходимо допустимий розв'язок:

$$z = 2 * 1 + 18 * 11 + 9 * 4 + 1 * (-6) - 3 * 9 = 203.$$

Розв'язок №2:

$$z = 203.$$

Висновок:

$$\min(315, 203) = 203.$$

Допустимий розв'язок задачі №2 кращий від допустимого розв'язку задачі №1.

Задача №4

Значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ для задачі на мінімум наведені у таблиці 1.10.

Таблиця 1.10 - Значення вхідних параметрів (задача на мінімум)

Назва масиву параметрів	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
a_j	11	3	3	-9	-9	15	19	13	-5	3
b_j	30	16	18	30	20	20	29	21	3	9

$$B = 140.$$

$$\sum_{j=1}^n b_j = 196 > B \text{ (задовольняє обмеження (6))}$$

У таблиці 1.11 наведений допустимий розв'язок №1 для задачі на мінімум.

Таблиця 1.11 - Допустимий розв'язок №1 (задача на мінімум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	23	12	15	-5	6	19	27	17	-3	5

Оскільки $\sum_{j=1}^n x_j = 116 \leq B$ (задовольняє обмеження (1)), то далі знаходимо допустимий розв'язок:

$$z = 23 * 12 + 15 * (-5) + 6 * 19 + 27 * 17 - 3 * 5 = 759.$$

Розв'язок №1:

$$z = 759.$$

У таблиці 1.12 наведений допустимий розв'язок №2 для задачі на мінімум.

Таблиця 1.12 - Допустимий розв'язок №2 (задача на мінімум)

	Порядковий елемент $j = 1, 2, \dots, n, n = 10$									
	1	2	3	4	5	6	7	8	9	10
x_j	22	12	18	5	6	18	23	15	-4	6

Оскільки $\sum_{j=1}^n x_j = 121 \leq B$ (задовольняє обмеження (1)), то далі знаходимо допустимий розв'язок:

$$z = 22 * 12 + 18 * 5 + 6 * 18 + 23 * 15 - 4 * 6 = 783.$$

Розв'язок №2:

$$z = 783.$$

Висновок:

$$\min (759, 783) = 759.$$

Допустимий розв'язок задачі №1 кращий від допустимого розв'язку задачі №2.

2. ОПИС МЕТОДІВ РОЗВ'ЯЗАННЯ ЗАДАЧІ

2.1 Найвні методи розв'язання

Наша задача відноситься до класу комбінаторних задач. Отже для дослідження та пошуку оптимального розв'язку можуть бути застосовані методи (алгоритми), що були розглянуті у курсі “Дослідження операцій” та “Інтелектуального аналізу даних”:

1. Методи локального пошуку

Алгоритми локального пошуку - це сім'я ітераційних методів, заснована на частковому перебиранні варіантів на кожній ітерації серед точок околу поточної точки, тобто серед сусідніх до неї. В алгоритмах цього типу замість повного перебору застосовується спрямований локальний перебір у підмножинах варіантів, які називаються околами. [1]

Загальна ідея - перебираючи околиці точок, знайти оптимальне значення шукаємої функції.

Мета методів - знаходження локальних екстремумів.

2. Генетичні алгоритми

Генетичний алгоритм - це еволюційний алгоритм пошуку, що використовується для розв'язання задач оптимізації, моделювання чи підтримки прийняття рішень шляхом послідовного відбору, комбінування й варіації шуканих параметрів з використанням механізмів, що нагадують біологічну еволюцію. [2]

Загальна ідея - перенесення принципу еволюційного відбору на процес оптимізації.

Мета таких алгоритмів - підвищення загального рівня пристосованості популяції.

3. Бджолиний алгоритм

Бджолиний алгоритм - один з поліноміальних евристичних алгоритмів для розв'язку оптимізаційних задач в області інформатики та дослідження операцій. Відноситься до категорії стохастичних біонічних алгоритмів, заснований на

імітації поведінки колонії медоносних бджіл при зборі нектару в природі. Ідея бджолиного алгоритму полягає в тому, що усі бджоли на кожному кроці будуть вибирати як елітні ділянки для дослідження, так і ділянки в околиці елітних, що дозволить, по-перше, урізноманітнити популяцію пошуків на наступних ітераціях, по-друге, збільшити ймовірність виявлення розв'язку близького до оптимального. [3]

Загальна ідея - збільшити ймовірність виявлення розв'язку близького до оптимального.

Мета методів - знаходження глобальних екстремумів (максимумів чи мінімумів) складних багатовимірних функцій.

4. Метод імітації відпалу

Метод імітації відпалу - стохастичний метод, призначений для вирішення завдань безумовної оптимізації та задач при наявності границь на змінні. Метод моделює процес нагрівання матеріального тіла з подальшим повільним зниженням температури для зменшення дефектів. [4]

Також існує варіант рішення задачі цим методом, якщо існують границі на змінні.

Загальна ідея - від зменшення температури горіння матеріального тіла збільшується ймовірність пошуку оптимального рішення.

Мета методу - пошук глобального екстремуму заданої функції.

Отже, обираючи алгоритми для розробки ми враховували складність методів (алгоритмів) та час, який знадобиться для розробки та впровадження відповідних алгоритмів.

Також варто зазначити, що Метод локального пошуку можна застосувати разом із Бджолиним алгоритмом, що покращить точність пошуку оптимального розв'язку. Метод імітації відпалу гарно підходить для розв'язку завдання курсової роботи, але цей метод є обчислювально складніший за Генетичний алгоритм.

Тому для розробки та впровадження були обрані саме Бджолиний алгоритм із застосуванням локального пошуку та Генетичний алгоритм.

2.2 Розробка жадібного алгоритму розв'язання задачі

1. Вхід:

$i := 1$ - ітератор алгоритму;
 n - кількість змінних;
 a_j ($j = 1, 2, \dots, n$)- вектор значень нижньої границі;
 b_j ($j = 1, 2, \dots, n$)- вектор значень верхньої границі;
 x_j ($j = 1, 2, \dots, n$) - допустимий вектор-розв'язок;
 B – константа.

2. Вихід:

LowestResult - мінімальне значення ЦФ;
HighestResult - максимальне значення ЦФ.

3. while $i \leq n$ do

4. **if** $(\sum_{j=1}^{i-1} x_j) + b_i \leq B$ **then** // Якщо сума знайдених змінних і максимально допустиме значення змінної (її верхня границя) на поточний ітерації не перевищує B . На першій ітерації сумування змінних не використовується, оскільки ітератор виходить за межі масиву.

5. $x_i := b_i$ // Присвоюємо змінній максимально допустиме значення (верхню границю поточного відрізка).

6. **else if** $B - \sum_{j=1}^{i-1} x_j \geq a_i$ **then** // Якщо залишок задовольняє мінімально допустиме значення.

7. $x_i := B - \sum_{j=1}^{i-1} x_j$ // Присвоюємо змінній максимально допустиме з значення, щоб сума змінних не перевищувала B .

8. **endif**

9. **endif**

10. $i := i + 1$

11. **endwhile**

12. *HighestResult* := підрахунок значення цільової функції

13. **while** $i \leq iterations$ **do**

14. **if** $(\sum_{j=1}^{i-1} x_j) + a_i \leq B$ **then** // Якщо сума знайдених змінних і мінімально допустиме значення змінної на поточний ітерації не перевищує B

15. $x_i := a_i$ // Присвоюємо змінній мінімально допустиме значення

16. $i := i + 1$

17. **endwhile**

18. *LowestResult* := підрахунок значення цільової функції

2.3 Опис бджолиного алгоритму із застосуванням локального пошуку

Бджолиний алгоритм відноситься до класу біологічних алгоритмів комбінаторної оптимізації. Цей алгоритм базується на результатах спостереження за бджолиними роями при пошуку нектару. Особливості поведінки бджолиних колоній можуть бути використані для моделювання інтелектуальної та колективної поведінки. [5]

У цьому алгоритмі бджоли-розвідники випадковим чином обирають ділянки для збору нектару, вилітаючи у різних напрямках із вулика. Після збору нектару на деякій ділянці бджоли повертаються до вулика та сповіщають інших про місця розташування нектару. Тоді бджоли-фуражири вирушають на вказані ділянки та збирають залишившийся нектар. [6]

Щоб адаптувати бджолиний алгоритм до нашої задачі було вирішено задати кількість бджіл-розвідників та бджіл-фуражирів однаковою (деякої розмірності n). Оскільки кількість ділянок в нас обмежена, то тут застосовується локальний пошук та оптимізацію в межах однієї із ітерацій алгоритму. Ці ділянки - околиці деяких шуканих точок x_j ($j = 1, 2, \dots, n$), мають вигляд відрізків $[a_j; b_j]$, до яких застосовуються обмеження (1)-(6).

На рисунку 2.1 зображений вигляд розташування ділянок для задачі 1 із розділу 1. На ці ділянки випадком чином прилітають бджоли-розвідники (рис. 2.2), а за ними бджоли-фуражири (рис. 2.3). Далі на кожній із ділянок збору нектару визначається користь від розташування бджоли та відбувається зміщення бджіл-розвідників на кращі ділянки (рис. 2.4) в залежності від завдання задачі. У даному випадку необхідно знайти максимальну суму попарних добутків точок розташування бджіл на ділянці-відрізку. Після цього відбувається перевірка допустимості (1) та за потребою обрані бджоли-розвідники змінюють своє розташування на допустиме (рис. 2.5).

Також варто зазначити, що обраний алгоритм є наближеним та на кожній ітерації знаходить допустимі розв'язки.

2.3.1 Розробка алгоритму розв'язання задачі

Загальна схема бджолиного алгоритму із застосуванням локального пошуку для задачі на максимум:

1. **Вхід:**
 k - загальна кількість ітерацій роботи алгоритму;
 $i := 1$ - ітератор алгоритму;
 n - кількість змінних;
 a_j ($j = 1, 2, \dots, n$)- вектор значень нижньої границі;
 b_j ($j = 1, 2, \dots, n$)- вектор значень верхньої границі;
 B – константа.
2. **Вихід:**
 $sumOpt$ - значення рекорду.
3. **while** $i \leq iterations$ **do**
4. **згенерувати** основний випадковий вектор-розв'язок
5. **локальне покращення** вектору-розв'язку
6. **підрахунок** поточного значення цільової функції
7. $sumOpt := \text{найбільше значення цільової функції}$ // Серед минулої та поточної ітерації. Якщо ітерація перша, то присвоїти значення поточної ітерації
8. $i := i + 1$ // Перехід до наступної ітерації алгоритму
9. **endwhile**
10. **return** $sumOpt$

Генерація випадкового вектору-розв'язку:

1. **Вхід:**
 a_j ($j = 1, 2, \dots, n$)- вектор значень нижньої границі;
 b_j ($j = 1, 2, \dots, n$)- вектор значень верхньої границі;
 B – константа.
2. **Вихід:**
 x_j ($j = 1, 2, \dots, n$) - допустимий вектор-розв'язок.
3. **while** розв'язок недопустимий **and** x_j повністю згенерований **do**
 //Перевірка допустимості
4. $x_j := \text{згенероване значення у діапазоні } [a_j; b_j]$
5. $j := j + 1$
6. **endwhile**
7. **return** x_j

Локальне покращення вектору-розв'язку:

1. **Вхід:**
 x_j ($j = 1, 2, \dots, n$) - допустимий вектор-розв'язок.
2. **Вихід:**

x_j ($j = 1, 2, \dots, n$) – покращений допустимий вектор-розв’язок.

3. згенерувати допоміжний допустимий вектор-розв’язок $xLocal_j$ ($j = 1, 2, \dots, n$)
4. **for** $j = 1$ **to** n **do**
5. **if** $x_j \geq 0$ **and** $x_{j+1} \geq 0$ **then** // Якщо обидва числа x_j та x_{j+1} одного добутку додатні
6. **if** $x_j < xLocal_j$ **then**
7. $x_j := xLocal_j$
8. **endif**
9. **endif**
10. **if** $x_j < 0$ **and** $x_{j+1} < 0$ **then** // Якщо обидва числа x_j та x_{j+1} одного добутку від’ємні
11. **if** $x_j > xLocal_j$ **then**
12. $x_j := xLocal_j$
13. **endif**
14. **endif**
15. **if** ($x_j < 0$ **and** $x_{j+1} \geq 0$) **or** ($x_j \geq 0$ **and** $x_{j+1} < 0$) **then** // Якщо одне із чисел x_j або x_{j+1} одного добутку додатне, а друге – від’ємне
16. **if** j - парний індекс **then** // Якщо обране друге із чисел добутку
17. **if** $x_j * x_{j-1} < xLocal_j * x_{j-1}$ **and** $(j - 1) \geq 0$ **then** // Якщо добуток від $xLocal_j$ на перше число з пари більше за добуток x_j на те саме число
18. $x_j := xLocal_j$ // Для x_j прийняти значення $xLocal_j$
19. **endif**
20. **else** // Інакше обране перше із чисел добутку
21. **if** $x_j * x_{j+1} < xLocal_j * x_{j+1}$ **and** $(j + 1) \geq n$ **then** // Якщо добуток від $xLocal_j$ на друге число з пари більше за добуток x_j на те саме число
22. $x_j := xLocal_j$
23. **endif**
24. **endif**
25. **endif**
26. $j := j + 1$ // Розглядаються лише числа в парі (парні добутки)
27. **endfor**
28. **if** розв’язок недопустимий **then** реанімація
29. **return** x_j

Реанімація:

1. **Вхід:**

x_j ($j = 1, 2, \dots, n$) – недопустимий покращений вектор-розв'язок.

2. **Вихід:**

x_j ($j = 1, 2, \dots, n$) - покращений допустимий вектор-розв'язок.

3. **while** розв'язок недопустимий **do**

4. вибрати два числа із x_j з найменшим добутком пар чисел

5. вибрати із чисел найбільше та зменшити його на 1

6. **endwhile**

7. **return** x_j

2.3.2 Оцінка трудомісткості бджолиного алгоритму із застосуванням локального пошуку

Складність обраного алгоритму - $O(n*k)$, де k – кількість ітерацій роботи алгоритму.

2.4 Опис Генетичного алгоритму

Генетичні алгоритми - це адаптивні методи пошуку, які використовуються для вирішення завдань оптимізації. У них використовуються як аналог механізму генетичного успадкування, так і аналог природного відбору. При цьому зберігається біологічна термінологія в спрощеному вигляді і основні поняття лінійної алгебри. [7]

Основою генетичного алгоритму є поняття популяції. Популяція це деяка сукупність допустимих значень змінних для конкретної задачі. Ціллю даного алгоритму є симуляція біологічного процесу тому, на кожній ітерації алгоритму з популяції обираються двоє найбільш пристосованих особин. Одним з підходів до вибору батьків є поділ популяції на дві підгрупи та вибір батьків по одному з кожної підгрупи, в яких дані особини мають найбільше значення функції пристосованості. Даний підхід називають турнірним відбором. [8]

За допомогою процесу рекомбінації вони утворюють двох нащадків. Принцип процесу рекомбінації може відрізнатись в залежності від задачі. Для прикладу, нижче відображено одне з правил для дискретної рекомбінації: Нащадок = Батько 1 + α * (Батько 1 – Батько 2).

В даній формулі коефіцієнт α може набувати будь якого значення але рекомендоване значення лежить в проміжку $[-0,25; 1,25]$. [9]

Над отриманими нащадками проводять процес мутації. Мутація - це певна видозміна нащадків за для набування ними особливих властивостей. На практиці використовується для того щоб нащадки мали можливість потрапити до інших околіїв.

Формула мутації для дискретних нащадків: Нова змінна = Стара змінна $\pm \alpha * \delta$, де знак $\pm$ означає що добуток коефіцієнтів α та δ набуває додатного чи від'ємного значення рівноймовірно. Коефіцієнт α дорівнює половині пошукового простору (інтервалу допустимої зміни для змінної). $\delta = \sum_{j=1}^m a(j)2^{-j}$, де $a(j)$ набуває значення 1 з імовірністю $\frac{1}{m}$, інакше 0. Параметр m може набувати будь якого значення. Рекомендоване значення для даного параметру дорівнює 20. З таким значенням m алгоритм мутації придатний до локалізації оптимуму з точністю $\alpha * 2^{-19}$. [10]

Після пройдених змін нащадки мають шанс бути недопустимими. В таких випадках необхідно провести процес реанімації. Недопустимими можуть бути значення змінних в генах нащадків. Якщо значення перевищує максимально допустиме, то значення змінної зменшується до максимально допустимого. Якщо значення применшує мінімально допустиме, то значення змінної збільшується до мінімально допустимого. Також недопустимою може бути значення суми змінних в генах нащадків. В таких випадках значення змінних рівномірно зменшується на 1 починаючи з максимального значення поки сума змінних не досягне допустимості.

Далі відбувається максимізація функції пристосованості за допомогою локальної оптимізації. Даний процес індивідуальний для кожної задачі. В даній задачі ціль оптимізації полягає в аналізі пар добутків.

При пошуку максимуму цільової функції пріоритетною задачею є максимізація в межах допустимості мінімальних значень на кожній ітерації. Винятком являються пари від'ємних добутків, та пари що мають обидва знаки. Від'ємні добутки необхідно навпаки мінімізувати межах допустимості і для

порівняння зі стандартним випадком беруться по модулю. Для пар добутків, що мають обидва знаки кожне значення необхідно максимізувати в межах допустимості. Для порівняння зі стандартним випадком від'ємні числа беруться без модулю. Процес повторюється поки не буде досягнуто ліміт по сумі змінних або всі змінні набудуть максимально допустимого значення.

При пошуку мінімуму цільової функції пріоритетною задачею є мінімізація в межах допустимості максимальних значень на кожній ітерації. Винятком являються пари від'ємних добутків, та пари що мають обидва знаки. Від'ємним добуткам необхідно надати значення максимально наближене до 0 і для порівняння зі стандартним випадком беруться по модулю. Для пар добутків, що мають обидва знаки, змінні з від'ємним знаком мінімізуються в межах допустимості, а змінні з додатнім максимізуються. Для порівняння зі стандартним випадком від'ємні числа беруться з протилежним знаком. Процес повторюється поки не буде досягнуто ліміт по сумі змінних або всі змінні набудуть мінімально допустимого значення.

Після локальної оптимізації відбувається оновлення популяції. Для задачі на максимум дві особини що мають найменше значення функції пристосованості замінюються на знайдених нащадків, якщо їхнє значення функції пристосованості більше. Для задачі на мінімум дві особини що мають найбільше значення функції пристосованості замінюються на знайдених нащадків, якщо їхнє значення функції пристосованості менше.

2.4.1 Розробка Генетичного алгоритму розв'язання задачі

Знаходження максимуму цільової функції

Схема роботи генетичного алгоритму:

1. Вхід:

k - загальна кількість ітерацій роботи алгоритму;

$i := 1$ - ітератор алгоритму;

n - кількість змінних;

a_j ($j = 1, 2, \dots, n$)- вектор значень нижньої границі;

b_j ($j = 1, 2, \dots, n$)- вектор значень верхньої границі;

B – константа.

2. **Вихід:**

Record - значення рекорду.

3. Генерація початкової популяції

4. **while** $i \leq k$ **do**

5. Турнірний відбір

6. Дискретна рекомбінація

7. Мутація

8. Реанімація

9. Локальна оптимізація

10. Оновлення популяції

11. *Record* := найбільше значення цільової функції // Значення порівнюється з попередніми ітераціями

12. $i := i + 1$ // Перехід до наступної ітерації

13. **endwhile**

14. **return** *Record*

Генерація початкової популяції:1. **Вхід:**

n - кількість особин;

m - кількість змінних;

a_j ($j = 1, 2, \dots, n$)- вектор значень нижньої границі;

b_j ($j = 1, 2, \dots, n$)- вектор значень верхньої границі;

B – константа.

2. **Вихід:**

x_{ij} ($i = 1, 2, \dots, n; j = 1, 2, \dots, m$) - популяція.

3. **for** $i = 1$ **to** n **do**4. **while** $\sum_{j=1}^m x_{ij} \leq B$ **do**5. **for** $j = 1$ **to** m **do**

6. $x_{ij} :=$ випадкове значення в діапазоні від $[a_j; b_j]$.

7. **endfor**8. **endwhile**9. **endfor**

10. **return** x

Турнірний відбір:1. **Вхід:**

n - кількість особин;

m - кількість змінних;

x_{ij} ($i = 1, 2, \dots, n; j = 1, 2, \dots, m$) - популяція;

$tournament_i$ ($i = 1, 2, \dots, m$)- булевий вектор для поділу популяції на групи.

2. **Вихід:**

$parent1_j$ ($j = 1, 2, \dots, m$)- вектор першого батька;
 $parent2_j$ ($j = 1, 2, \dots, m$)- вектор другого батька;
3. for $i = 1$ **to** n **do**
 4. $tournament_i :=$ випадкове значення в діапазоні від $[0; 1]$. // Для поділу популяції на дві групи
 5. **endfor**
 6. **for** $i = 1$ **to** n **do**
 7. **if** $tournament_i = 0$ **and** $\sum_{j=2k-1, k \in [1.. \frac{m}{2}]} x_{ij} * x_{ij+1} >$
 $\sum_{j=2k-1, k \in [1.. \frac{m}{2}]} parent1_j * parent1_{j+1}$ **then** // Відбір особини що має
 максимальне значення ЦФ в групі 0
 8. $parent1 := x_i$
 9. **else if** $tournament_i = 1$ **and** $\sum_{j=2k-1, k \in [1.. \frac{n}{2}]} x_{ij} * x_{ij+1} >$
 $\sum_{j=2k-1, k \in [1.. \frac{n}{2}]} parent2_j * parent2_{j+1}$ **then** // Відбір особини, що має
 максимальне значення ЦФ в групі 1
 10. $parent2 := x_i$
 11. **endif**
 12. **endfor**
 13. **return** $parent1, parent2$

Дискретна рекомбінація:

1. Вхід:
 n - кількість змінних;
 $parent1_j$ ($j = 1, 2, \dots, n$)- вектор першого батька;
 $parent2_j$ ($j = 1, 2, \dots, n$)- вектор другого батька;
 α - коефіцієнт.
2. Вихід:
 $child1_j$ ($j = 1, 2, \dots, n$)- вектор першого нащадка;
 $child2_j$ ($j = 1, 2, \dots, n$)- вектор другого нащадка.
3. for $j = 1$ **to** n **do**
 4. $\alpha :=$ випадкове значення в діапазоні від $[-0,25; 1,25]$.
 5. $child1_j := \alpha(parent2_j - parent1_j)$
 6. $\alpha :=$ випадкове значення в діапазоні від $[-0,25; 1,25]$.
 7. $child2_j := \alpha(parent2_j - parent1_j)$
 8. **endfor**
 9. **return** $child1, child2$

Мутація:

1. Вхід:
 n - кількість змінних;

a_j ($j = 1, 2, \dots, m$)- вектор значень нижньої границі;
 b_j ($j = 1, 2, \dots, m$)- вектор значень верхньої границі;
 $child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;
 $child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка;
 k, α, δ - коефіцієнти;
 m - параметр.

2. Вихід:

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;
 $child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка.

3. for $j = 1$ to n do

4. $\alpha := \frac{b_j - a_j}{2}$

5. $\delta := \sum_{j=1}^m a(j) 2^{-j} // a(j) = 1$ з імовірністю $\frac{1}{m}$, інакше 0.

6. $child1_j := child1_j \pm \alpha * \delta // \pm$ означає, що добуток рівноймовірно набуває або додатного або від'ємного значення.

7. endfor

8. for $j = 1$ to n do

9. $\alpha := \frac{b_j - a_j}{2}$

10. $\delta := \sum_{j=1}^m a(j) 2^{-j} // a(j) = 1$ з імовірністю $\frac{1}{m}$, інакше 0.

11. $child2_j := child2_j \pm \alpha * \delta // \pm$ означає, що добуток рівноймовірно набуває або додатного або від'ємного значення.

12. endfor

13. return $child1, child2$

Реанімація:

1. Вхід:

n - кількість змінних;
 a_j ($j = 1, 2, \dots, m$)- вектор значень нижньої границі;
 b_j ($j = 1, 2, \dots, m$)- вектор значень верхньої границі;
 $child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;
 $child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка;
 B – константа;
 max - максимальне значення змінної;
 $maxIndex$ - індекс змінної з максимальним значенням.

2. Вихід:

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;
 $child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка.

3. for $j = 1$ to n do

4. **if $child1_j > b_j$ then** // Якщо значення більше за максимальне,

```

5.       $child1_j := b_j$  // Надання змінній значення максимально
      допустимого значення.
6.      else if  $child1_j < a_j$  then // Якщо значення менше за мінімальне,
7.       $child1_j := a_j$  // Надання змінній значення мініально
      допустимого значення.
8.      endif
9. endfor
10. while  $\sum_{j=1}^n child1_j > B$  do // Поступове зменшення максимальних
      значень для отримання допустимості
11.   for  $j = 1$  to  $n$  do
12.     if  $child1_j > max$  then
13.        $max := child1_j$ 
14.        $maxIndex := j$ 
15.     endif
16.   endfor
17.    $child_{maxIndex} := child_{maxIndex} - 1$ 
18. for  $j = 1$  to  $n$  do
19.   if  $child2_j > b_j$  then // Якщо значення більше за максимальне,
20.    $child2_j := b_j$  // Надання змінній значення максимально
      допустимого значення.
21.   else if  $child2_j < a_j$  then // Якщо значення менше за мінімальне,
22.    $child2_j := a_j$  // Надання змінній значення мініально
      допустимого значення.
23.   endif
24. endfor
25. while  $\sum_{j=1}^n child2_j > B$  do // Поступове зменшення максимальних
      значень для отримання допустимості
26.   for  $j = 1$  to  $n$  do
27.     if  $child2_j > max$  then
28.        $max := child2_j$ 
29.        $maxIndex := j$ 
30.     endif
31.   endfor
32.    $child_{maxIndex} := child_{maxIndex} - 1$ 
33. endwhile
34. return  $child1, child2$ 

```

Локальна оптимізація:

1. Вхід:

n - кількість змінних;

a_j ($j = 1, 2, \dots, m$)- вектор значень нижньої границі;

b_j ($j = 1, 2, \dots, m$)- вектор значень верхньої границі;

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка

B – константа.

2. **Вихід:**

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка.

3. **while** $\sum_{j=1}^n child1_j = B$ **or** всі змінні мають максимальне значення **do**

4. Ціль оптимізації полягає в аналізі пар добутоків.

Пріоритетною задачею є максимізація межах допустимості мінімальних значень на кожній ітерації. Винятком являються пари від'ємних добутоків, та пари що мають обидва знаки.

Від'ємні добутки необхідно навпаки мінімізувати межах допустимості і для порівняння зі стандартним випадком беруться по модулю.

Для пар добутоків, що мають обидва знаки кожне значення необхідно максимізувати в межах допустимості. Для порівняння зі стандартним випадком від'ємні числа беруться без модулю.

5. **endwhile**

6. **while** $\sum_{j=1}^n child2_j = B$ **or** всі змінні мають максимальне значення **do**

7. Справедливі ідентичні судження до тих що описані вище у кроці 4.

8. **endwhile**

9. **return** $child1, child2$

Оновлення популяції:

1. **Вхід:**

n - кількість особин;

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка

B – константа.

2. **Вихід:**

x_{ij} ($i = 1, 2, \dots, n; j = 1, 2, \dots, m$) - популяція.

3. **for** $i = 1$ **to** n **do**

4. Пошук двох особини з мінімальним значенням цільової функції

5. Замінити їх знайденими нащадками $child1, child2$, якщо вони мають більше значення цільової функції

6. **endfor**

Знаходження мінімуму цільової функції

Процес роботи алгоритму ідентичний задачі на максимум за винятком локальної оптимізації та оновлення популяції, що описані нижче.

Локальна оптимізація:

1. Вхід:

n - кількість змінних;

a_j ($j = 1, 2, \dots, m$)- вектор значень нижньої границі;

b_j ($j = 1, 2, \dots, m$)- вектор значень верхньої границі;

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка;

B – константа;

2. Вихід:

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка;

3. **while** $\sum_{j=1}^n child1_j = B$ **or** всі змінні мають максимальне значення **do**

4. Ціль оптимізації полягає в аналізі пар добутоків.

Пріоритетною задачею є мінімізація межах допустимості максимальних значень на кожній ітерації. Винятком являються пари від'ємних добутоків, та пари що мають обидва знаки.

Від'ємним добуткам необхідно надати значення максимально наближене до 0 і для порівняння зі стандартним випадком беруться по модулю.

Для пар добутоків, що мають обидва знаки, змінні з від'ємним знаком мінімізуються межах допустимості, а змінні з додатнім максимізуються. Для порівняння зі стандартним випадком від'ємні числа беруться з протилежним знаком.

5. **endwhile**

6. **while** $\sum_{j=1}^n child2_j = B$ **or** всі змінні мають максимальне значення **do**

7. Справедливі ідентичні судження до тих, що описані вище у кроці 4.

8. **endwhile**

9. **return** $child1, child2$

Оновлення популяції:

1. Вхід:

n - кількість особин;

$child1_j$ ($j = 1, 2, \dots, m$)- вектор першого нащадка;

$child2_j$ ($j = 1, 2, \dots, m$)- вектор другого нащадка

B – константа.

2. Вихід:

x_{ij} ($i = 1, 2, \dots, n; j = 1, 2, \dots, m$) - популяція.

3. **for** $i = 1$ **to** n **do**

4. Пошук двох особини з мінімальним значенням цільової функції.

5. Замінити їх знайденими нащадками $child1, child2$, якщо вони мають більше значення цільової функції

6. endfor

2.4.2 Оцінка трудомісткості алгоритму Генетичного алгоритму

Складність алгоритму - $O(n^2)$, оскільки найбільш трудомісткою операцією є локальна оптимізація, що виконує пошук максимального (для задачі на мінімум) та мінімального (для задачі на максимум) елемента, що потребує використання 2-х циклів.

2.5 Приклади застосування розроблених алгоритмів

2.5.1 Приклад розв'язання жадібним алгоритмом

Дано:

- $n=10$;
- Вхідні значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ наведені в таблиці 1.1.;
- $B=115$.

Знайти: Значення і максимуму *HighestResult* і мінімуму *LowestResult* цільової функції.

Розв'язання:

Знаходження максимуму цільової функції.

Як продемонстровано в таблиці 2.1 на кожній наступній ітерації жадібний алгоритм присвоює поточний змінній максимально допустиме значення. Поки справедливе обмеження на суму змінних, то максимально допустиме значення для змінних це значення відповідних елементів з вектору b . Коли значення суми змінних перевищує допустиме з обмеження про максимальну суму, то береться максимальне число, яке буде допустимим для даного обмеження.

В прикладі, зображеному в таблиці 2.1, обмеження про максимальну суму було досягнуто на ітерації 10, саме тому для змінної x_j було обрано значення 9 хоч максимально допустиме значення без обмеження на суму змінних дорівнює 20.

Таблиця 2.1 - Знаходження мінімуму ЦФ з використанням жадібного алгоритму

Ітерації	1	2	3	4	5	6	7	8	9	10	Сума	Значення ЦФ
1	10										10	0
2	10	19									29	190
3	10	19	22								51	190
4	10	19	22	11							62	432
5	10	19	22	11	1						63	432
6	10	19	22	11	1	10					73	442
7	10	19	22	11	1	10	13				86	442
8	10	19	22	11	1	10	13	19			105	689
9	10	19	22	11	1	10	13	19	1		106	689
10	10	19	22	11	1	10	13	19	1	9	115	698

Розв'язок: HighestResult = 698.

Знаходження мінімуму цільової функції

Як продемонстровано в таблиці 2.2 на кожній наступній ітерації жадібний алгоритм присвоює поточний змінній мінімально допустиме значення. Поки справедливе обмеження на суму змінних, то мінімально допустиме значення для змінних це значення відповідних елементів з вектору а. Поки значення суми змінних не перевищує допустиме з обмеження про максимальну суму, то береться мінімальне число.

Таблиця 2.2 - Знаходження мінімуму ЦФ з використанням жадібного алгоритму

Ітерації	1	2	3	4	5	6	7	8	9	10	Сума	Значення ЦФ
1	-14										-14	0
2	-14	12									-2	-168
3	-14	12	10								8	-168
4	-14	12	10	0							8	-168
5	-14	12	10	0	-24						-16	-168
6	-14	12	10	0	-24	3					-13	-240
7	-14	12	10	0	-24	3	13				0	-240
8	-14	12	10	0	-24	3	13	7			7	-149
9	-14	12	10	0	-24	3	13	7	-13		-6	-149
10	-14	12	10	0	-24	3	13	7	-13	2	-4	-175

Розв'язок: LowestResult = -175.

2.5.2 Приклад розв'язання задачі на максимум бджолиним алгоритмом із застосуванням локального пошуку

Дано:

- $n=10$;
- Вхідні значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ наведені в таблиці 1.1.;

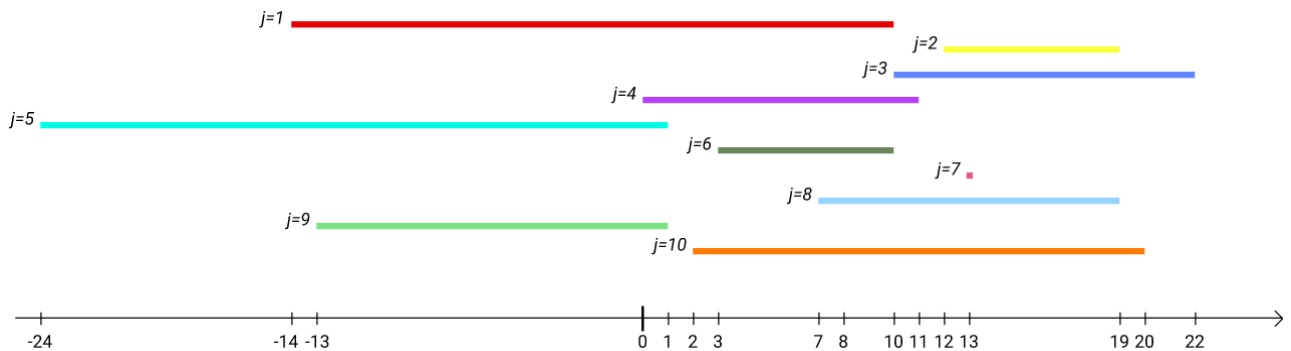


Рис. 2.1 - Розташування ділянок для збору нектару

- $B=70$;
- $k=2$.

Знайти: $sumOpt$ - значення цільової функції відповідно до формули (7).

Розв'язання:

Ітерація 1

1. Генеруємо допустимий випадковий розв'язок

У таблиці 2.3 наведено значення згенерованого вектора-розв'язка x_j .

Також виконана перевірка на допустимість наступним чином:

$$\sum_{j=1}^n x_j = -4 + 17 + 13 + 1 - 8 + 9 + 13 + 11 - 2 + 15 = 65 \leq B.$$

Для порівняння наступних отриманих результатів було підраховане значення цільової функції, що також вказане у таблиці 2.3.

Таблиця 2.3 - Згенерований допустимий випадковий розв'язок

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n x_j$
x_j	-4	17	13	1	-8	9	13	11	-2	15	-14	65

На рисунку 2.2 проілюстровані позиції бджіл-розвідників, що були згенеровані у таблиці 2.3.

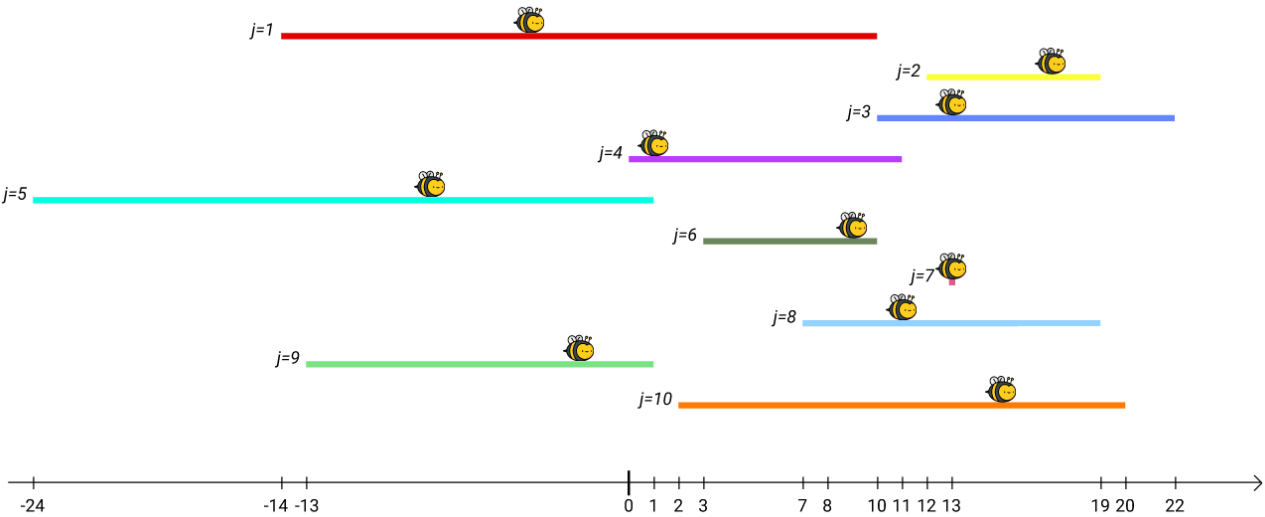


Рис. 2.2 - Розташування бджіл-розвідників на відповідних ділянках

2. Виконуємо локальне покращення

Для локального покращення ми застосовуємо окіл точок (нижню і верхню границю a_j та b_j відповідного j). У заданому околі для кожного j генеруємо нові допустимі розв’язки (новий допоміжний допустимий розв’язок - $xLocal$) (табл. 2.4):

Таблиця 2.4 - Згенерований допоміжний допустимий випадковий розв’язок

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n xLocal_j$
$xLocal_j$	0	12	19	1	-6	4	13	16	-13	16	-5	62

На рисунку 2.3 блакитними є бджоли-фуражири, розташування яких вказане у таблиці 2.4.

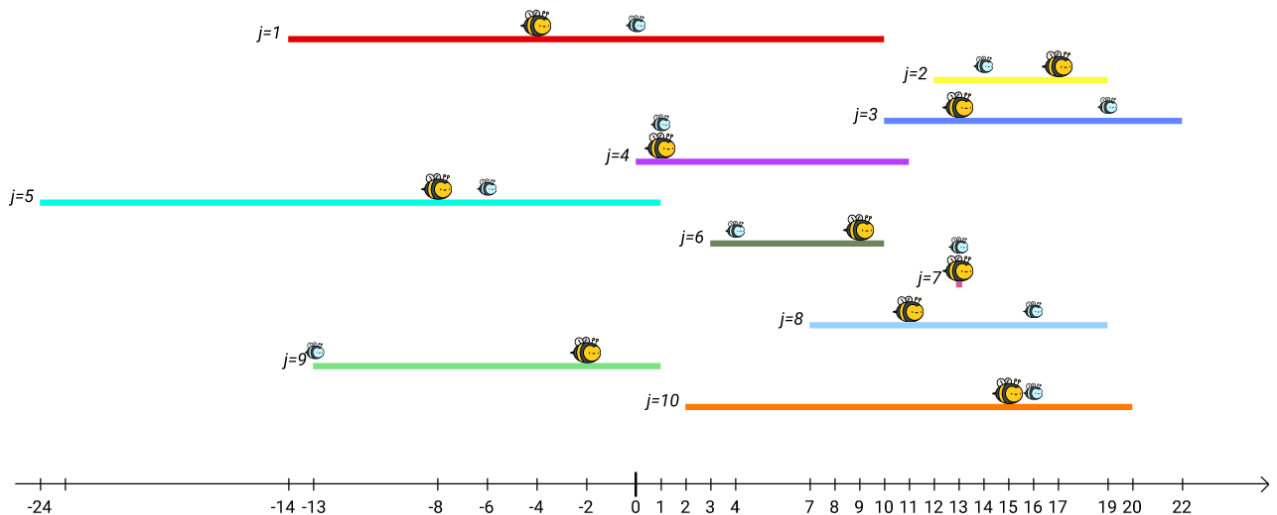


Рис. 2.3 - Розташування бджіл-розвідників та бджіл-фуражирів на відповідних ділянках

Розглядаємо такі випадки для добутків пар чисел (табл. 2.5):

- 1) Обидва із чисел добутка додатні числа. Позначаються в таблиці як “+” та “+”. Тут поточний x_j приймає значення $xLocal_j$, якщо $xLocal_j$ більший за x_j .
- 2) Обидва із чисел добутка від’ємні числа. Позначаються в таблиці як “-” та “-”. Тут поточний x_j приймає значення $xLocal_j$, якщо $xLocal_j$ менший за x_j .
- 3) Одне із чисел добутка має додатне значення, інше - від’ємне. Позначаються в таблиці як “+” та “-” або навпаки. Тут поточний x_j приймає значення $xLocal_j$, якщо добуток від $xLocal_j$ на друге число з пари більше за добуток x_j на друге число з відповідної пари.

Таблиця 2.5 - Добутки та значення знаків у парі чисел сновного розв’язку

-	+	+	+	-	+	+	+	-	+
---	---	---	---	---	---	---	---	---	---

Таким чином отримуємо покращені значення вектору-розв’язку x_j , що наведені у таблиці 2.6. Змінені x_j позначені зеленим кольором. Але водночас не виконується умова допустимості:

$$\sum_{j=1}^n x_j = 0 + 17 + 19 + 1 - 6 + 4 + 13 + 16 - 2 + 15 = 77 > B.$$

Таблиця 2.6 - Локальне покращення

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n x_j$
x_j	0	17	19	1	-6	4	13	16	-2	15	170	77

На рисунку 2.4 зелені бджоли - бджоли-розвідники, що зайняли позиції бджіл-фуражирів з рисунку 2.3.

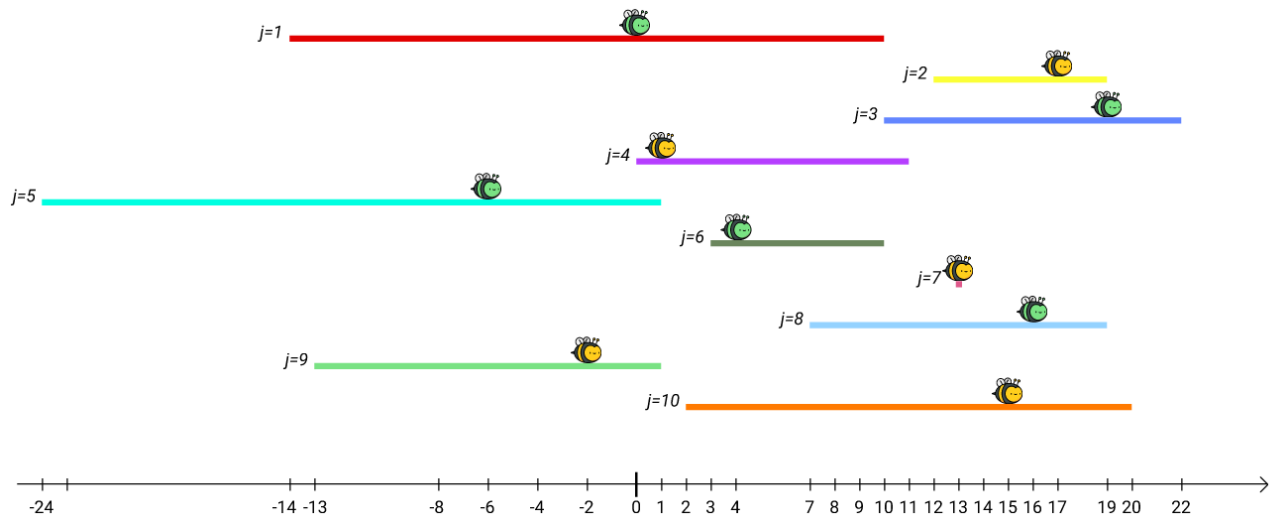


Рис. 2.4 - Зміщення бджіл-розвідників на кращі позиції на ділянці

Отже маємо перейти до реанімування, щоб досягти умови допустимості.

3. Реанімуємо

Обираємо найменший із добутків та починаємо зменшувати найбільше із чисел добутка. Початковий стан вибору найменшого добутка наведений у таблиці 2.7.

Таблиця 2.7 - Добутки та значення чисел у парі (недопустимий розв'язок)

17		19		-24		208		-30	
0	17	19	1	-6	4	13	16	-2	15

У таблиці 2.8 червоним кольором виділені ті числа, що були зменшені для досягнення допустимості.

Таблиця 2.8 - Добутки та значення чисел у парі (допустимий розв'язок)

10		19		-18		208		-18	
0	17	19	1	-6	3	13	16	-2	9

На рисунку 2.5 червоні бджоли - бджоли-розвідники, що були реанімовані та змінили свої позиції на допустимі.

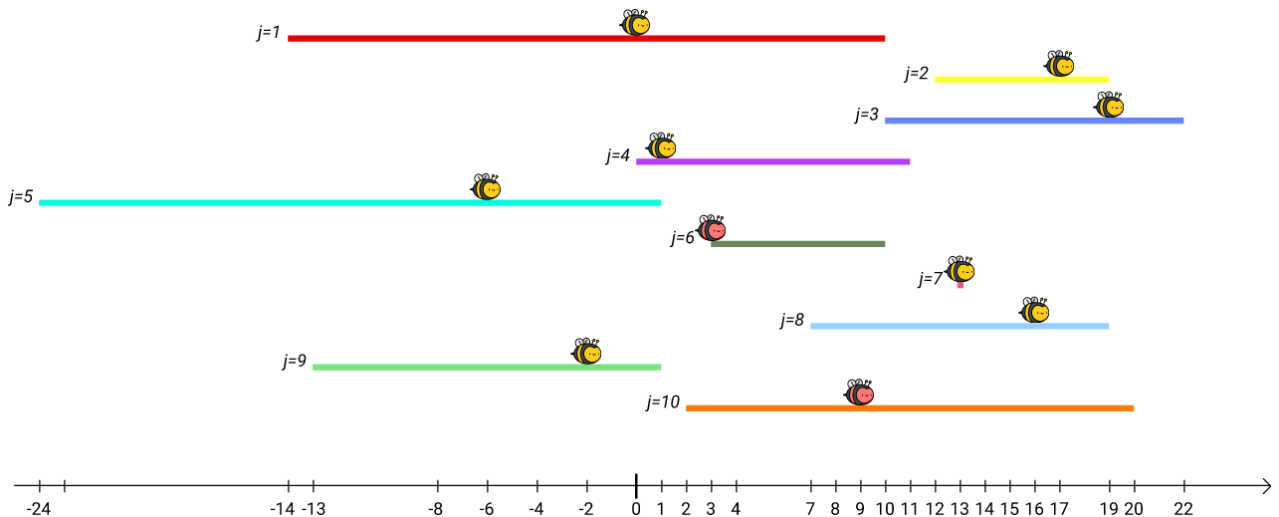


Рис. 2.5 - Зміщення бджіл-розвідників на кращі допустимі позиції на ділянці

Тепер умова допустимості виконується:

$$\sum_{j=1}^n x_j = 0 + 17 + 19 + 1 - 6 + 3 + 13 + 16 - 2 + 9 = 77 \leq B.$$

4. Підраховуємо значення цільової функції поточної ітерації

За умов допустимості отримуємо нове значення цільової функції:

$$result = 0 * 17 + 19 * 1 + (-6) * 3 + 13 * 16 + (-2) * 9 = 191.$$

5. Визначаємо рекорд

На першій ітерації рекордом буде його перше знайдене значення: $sumOpt = result = 191$.

Ітерація 2

1. Генеруємо допустимий випадковий розв'язок

У таблиці 2.9 наведено значення згенерованого вектора-розв'язка x_j . Також виконана перевірка на допустимість наступним чином:

$$\sum_{j=1}^n x_j = 3 + 12 + 11 + 2 + 0 + 7 + 13 + 15 - 10 + 16 = 69 \leq B.$$

Для порівняння наступних отриманих результатів було підраховане значення цільової функції, що також вказане у таблиці 2.9.

Таблиця 2.9 - Згенерований допустимий випадковий розв'язок

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n x_j$
x_j	3	12	11	2	0	7	13	15	-10	16	93	69

На рисунку 2.6 проілюстровані позиції бджіл-розвідників, що були згенеровані у таблиці 2.9.

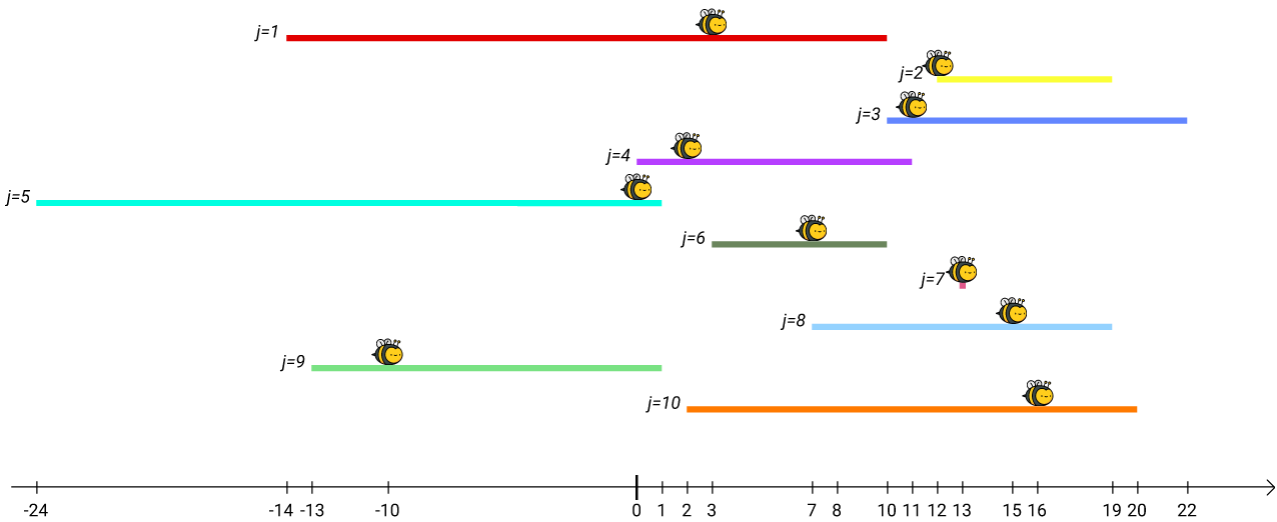


Рис. 2.6 - Розташування бджіл-розвідників на відповідних ділянках

2. Виконуємо локальне покращення

Для локального покращення ми застосовуємо окіл точок (нижню і верхню границю a_j та b_j відповідного j). У заданому околі для кожного j генеруємо нові допустимі розв’язки (новий допоміжний допустимий розв’язок - $xLocal$) (табл. 2.10):

Таблиця 2.10 - Згенерований допоміжний допустимий випадковий розв’язок

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n xLocal_j$
$xLocal_j$	-6	13	10	3	-11	8	13	13	-1	13	20	55

На рисунку 2.7 блакитними є бджоли-фуражири, розташування яких вказане у таблиці 2.10.

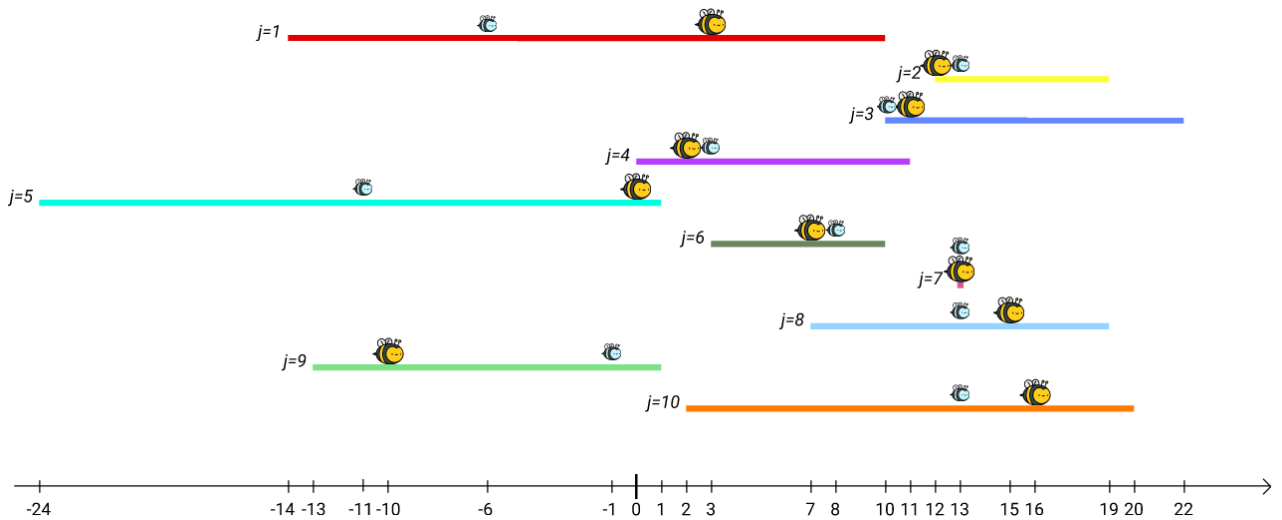


Рис. 2.7 - Розташування бджіл-розвідників та бджіл-фуражирів на відповідних ділянках

Розглядаємо знаки у добутках пар чисел (табл. 2.11):

Таблиця 2.11 - Добутки та значення знаків у парі чисел основного розв'язку

-	+	+	+	-	+	+	+	-	+
---	---	---	---	---	---	---	---	---	---

Таким чином отримуємо покращені значення вектору-розв'язку x_j , що наведені у таблиці 2.12. Змінені x_j позначені зеленим кольором. Але водночас не виконується умова допустимості:

$$\sum_{j=1}^n x_j = 3 + 13 + 11 + 3 + 0 + 8 + 13 + 15 - 1 + 13 = 78 > B.$$

Таблиця 2.12 - Локальне покращення

j	1	2	3	4	5	6	7	8	9	10	ЦФ	$\sum_{j=1}^n x_j$
x_j	3	13	11	3	0	8	13	15	-1	13	267	78

На рисунку 2.8 зелені бджоли - бджоли-розвідники, що зайняли позиції бджіл-фуражирів з рисунку 2.7.

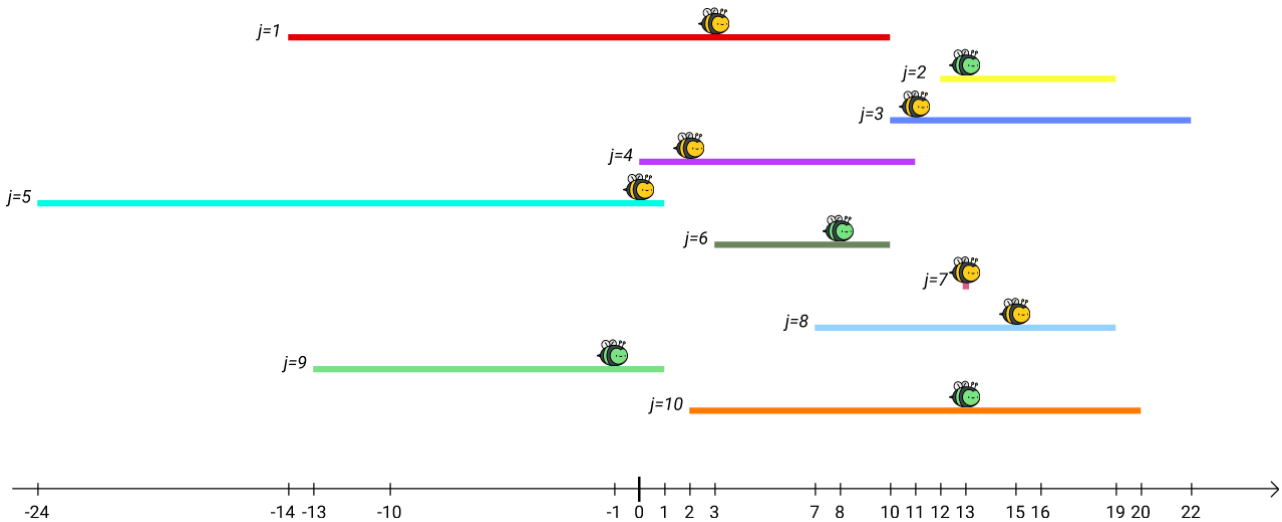


Рис. 2.8 - Зміщення бджіл-розвідників на кращі позиції на ділянці

Отже маємо перейти до реанімування, щоб досягти умови допустимості.

3. Реанімуємо

Обираємо найменший із добутків та починаємо зменшувати найбільше із чисел добутка. Початковий стан вибору найменшого добутка наведений у таблиці 2.13.

Таблиця 2.13 - Добутки та значення чисел у парі (недопустимий розв'язок)

39		33		0		195		-13	
3	13	11	3	0	8	13	15	-1	13

У таблиці 2.14 червоним кольором виділені ті числа, що були зменшені для досягнення допустимості.

Таблиця 2.14 - Добутки та значення чисел у парі (допустимий розв'язок)

39		33		0		195		-5	
3	13	11	3	0	8	13	15	-1	5

На рисунку 2.9 червоні бджоли - бджоли-розвідники, що були реанімовані та змінили свої позиції на допустимі.

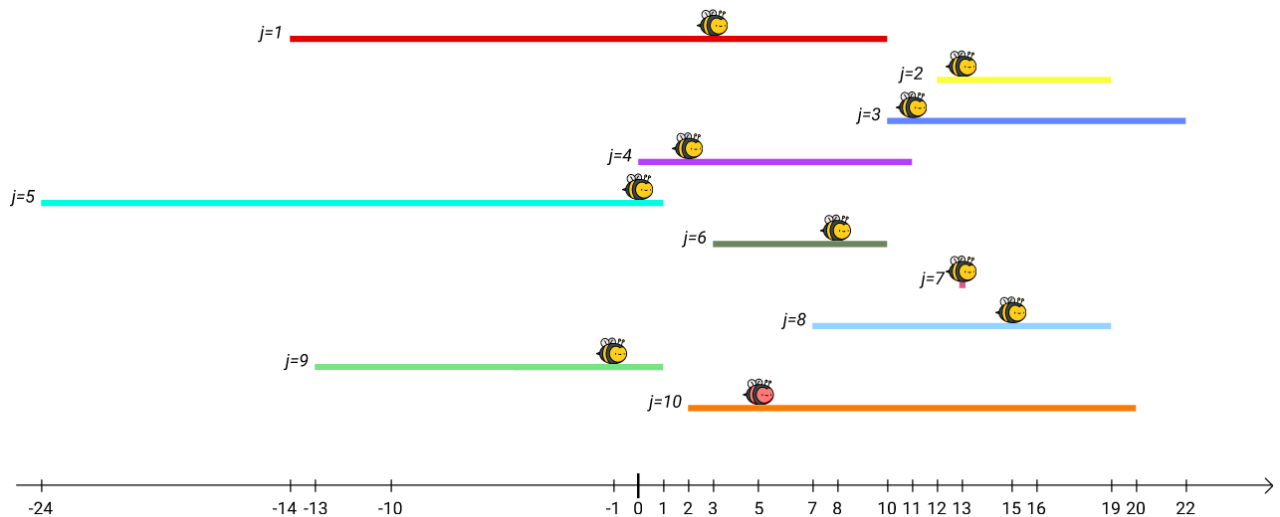


Рис. 2.9 - Зміщення бджіл-розвідників на кращі допустимі позиції на ділянці

Тепер умова допустимості виконується:

$$\sum_{j=1}^n x_j = 3 + 13 + 11 + 3 + 0 + 8 + 13 + 15 - 1 + 5 = 70 \leq B.$$

4. Підраховуємо значення цільової функції поточної ітерації

За умов допустимості отримуємо нове значення цільової функції:

$$result = 3 * 13 + 11 * 3 + 0 * 8 + 13 * 15 + (-1) * 5 = 262.$$

5. Визначаємо рекорд

На другій ітерації рекордом буде його поточне значення цільової функції, оскільки результат знайдений на першій ітерації менше за результат другої ітерації:

$$sumOpt = result = 262.$$

$$Розв'язок: sumOpt = 262.$$

2.5.3 Приклад розв'язання задачі Генетичним алгоритмом

Дано:

- $n=10$;
- Вхідні значення параметрів $a_j, b_j, j = 1, 2, \dots, n$ наведені в таблиці 1.1.;
- $B=115$.

Знайти: Значення мінімуму і максимуму цільової функції.

Розв'язання:

Знаходження максимуму цільової функції.

1. Генерація популяції із 15-ти особин.

Поділ особин на 2 групи випадковим чином (у табл. 2.15 зеленим кольором виділена перша група особин, оранжевим – друга). Згенеровані значення зображені в таблиці 2.15. Значення цільової функції для згенерованих нащадків знаходиться по формулі $\sum_{j=2k-1, k \in [1.. \frac{m}{2}]} x_{ij} * x_{ij+1}$, де $m = 10$.

Таблиця 2.15 - Початкова популяція

Група	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	-10	12	10	6	-1	5	13	11	0	12	58	78
1	7	16	14	7	-8	9	13	7	-8	10	67	149
0	-11	18	13	8	-8	5	13	11	-10	14	53	-131
0	-4	14	14	11	0	8	13	18	-12	5	67	272
1	9	12	18	7	-10	6	13	18	-3	6	76	390
1	-5	18	19	7	0	7	13	10	-10	6	65	113
0	1	12	15	4	-17	4	13	17	0	12	61	225
0	-1	19	17	10	-16	6	13	16	-9	11	66	164
0	4	15	21	1	-1	7	13	11	-13	15	73	22
0	-6	16	17	6	-20	3	13	8	0	3	40	50
1	1	12	19	3	-16	10	13	9	-6	19	64	-88
0	-12	16	17	6	-2	6	13	18	-2	5	65	122
1	-6	18	11	4	-2	8	13	13	0	3	62	89
1	0	13	17	11	-23	10	13	16	-9	5	53	120
1	4	13	10	11	0	4	13	7	-4	3	61	241

Ітерація 1

2. Турнірний відбір

Вибір по одній особині з кожної групи, що мають найбільше значення цільової функції (виділено фіолетовим кольором). Вибрані особини відображені в таблиці 2.16.

Таблиця 2.16 - Гени батьків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Батько 1	-4	14	14	11	0	8	13	18	-12	5	67	272
Батько 2	9	12	18	7	-10	6	13	18	-3	6	76	390

3. Дискретна рекомбінація

Значення змінної знаходиться як значення відповідної змінної першого батька * α * (значення відповідної змінної другого батька - значення відповідної змінної першого батька). Для першого нащадка значення коефіцієнту $\alpha = 0,5$. Для другого $\alpha = 0,1$. Знайдені гени нащадків відображені в таблиці 2.17.

Таблиця 2.17 - Гени нащадків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	3	13	16	9	-5	7	13	18	-8	6	72	334
Нащадок 2	-3	14	14	11	-1	8	13	18	-11	5	68	285

4. Мутація

Мутація відбувається по формулі: *нове значення змінної* = *старе значення змінної* $\pm \alpha * \delta$, де знак $\pm$ означає, що добуток набуває або від'ємного або додатного значення (табл. 2.18); α - половина проміжку для допустимої зміни значення змінної; $\delta = \sum_{j=1}^m a(j)2^{-j}$, де $a(j) = 1$ з імовірністю $\frac{1}{m}$, інакше 0.

Для даної задачі ($m = 20$) значення коефіцієнтів відображені в таблиці 2.18. Гени нащадків після мутації відображені в таблиці 2.19.

Таблиця 2.18 - Значення коефіцієнтів для мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
$\pm$	+	+	+	--	+	-	-	+	-	+
	-	+	+	+	-	-	+	-	-	-
α	12	3,5	6	5,5	12,5	3,5	0	6	7	11
δ	0,70	0,18	0,42	0,33	0,47	0,78	0,56	0,47	0,32	0,70
	0,86	0,51	0,28	0,18	0,92	0,02	0,49	0,21	0,12	0,68

5. Реанімація

Проведення реанімації для генів нащадків, що мають недопустимі значення змінних. Недопустимі значення позначені червоним кольором в таблиці 2.19.

Таблиця 2.19 - Гени нащадків після мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Нащадок 1	11	14	19	7	1	4	13	21	-10	13
Нащадок 2	-13	16	16	12	-13	8	13	17	-12	-2

Перший нащадок має 2 недопустимі змінні та другий нащадок має 1 недопустиму змінну, що перевищують максимальне допустиме значення значення. Для отримання допустимості дані змінні необхідно зменшити до максимально допустимих. Отримані значення відображенні зеленим кольором в таблиці 2.20.

Таблиця 2.20 - Гени нащадків після реанімації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	10	14	19	7	1	4	13	19	-10	13	90	391

Нащадок 2	-13	16	16	11	-13	8	13	17	-12	-2	40	123
-----------	-----	----	----	----	-----	---	----	----	-----	----	----	-----

6. Локальна оптимізація

Пріоритетність на максимізацією елементів залишається за змінними з мінімальним значенням. Винятками є остання пара змінних x_9 та x_{10} (таблиця 2.21 та 2.22). Оскільки це пара змінних з від'ємним знаком, то їхні значення мінімізуються.

Таблиця 2.21 - Гени під час локальної оптимізації першого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	10	14	19	7	1	4	13	19	-10	-2	75	539
2	10	14	19	7	1	10	13	19	-10	-2	80	544
3	10	14	19	11	1	10	13	19	-10	-2	84	615
4	10	14	19	11	1	10	13	19	-13	-2	81	621
5	10	19	19	11	1	10	13	19	-13	-2	86	675
6	10	19	22	11	1	10	13	19	-13	-2	90	715

Таблиця 2.22 - Гени під час локальної оптимізації другого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	10	16	16	11	-13	8	13	17	-12	-2	64	478
2	10	16	16	11	1	8	13	17	-12	-2	77	582
3	10	16	16	11	1	10	13	17	-12	-2	80	584
4	10	16	16	11	1	10	13	17	-13	-2	78	587
5	10	19	16	11	1	10	13	17	-13	-2	82	621
6	10	19	22	11	1	10	13	17	-13	-2	88	686
7	10	19	22	11	1	10	13	19	-13	-2	90	715

7. Оновлення популяції

Необхідно вибрати 2 особи з найменшим значенням цільової функції (позначені сірим кольором в таблиці 2.15) та замінити їх на знайдені нащадки (позначені жовтим кольором в таблиці 2.23). Поділ особин на 2 групи випадковим чином.

Таблиця 2.23 - Популяція оновлених особин

Група	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
0	-10	12	10	6	-1	5	13	11	0	12	58	78
0	7	16	14	7	-8	9	13	7	-8	10	67	149
1	10	19	22	11	1	10	13	19	-13	-2	90	715
0	-4	14	14	11	0	8	13	18	-12	5	67	272
1	9	12	18	7	-10	6	13	18	-3	6	76	390
1	-5	18	19	7	0	7	13	10	-10	6	65	113
1	1	12	15	4	-17	4	13	17	0	12	61	225
0	-1	19	17	10	-16	6	13	16	-9	11	66	164

0	4	15	21	1	-1	7	13	11	-13	15	73	22
1	-6	16	17	6	-20	3	13	8	0	3	40	50
1	10	19	22	11	1	10	13	19	-13	-2	90	715
1	-12	16	17	6	-2	6	13	18	-2	5	65	122
0	-6	18	11	4	-2	8	13	13	0	3	62	89
0	0	13	17	11	-23	10	13	16	-9	5	53	120
1	4	13	10	11	0	4	13	7	-4	3	61	241

8. Визначення рекорду

На першій ітерації рекордом буде максимальне значення цільової функції серед усіх особин популяції, тому значення змінної:

$Record = 715$.

Ітерація 2

1. Турнірний відбір

Вибір по одній особині з кожної групи з таблиці 2.23, що мають найбільше значення цільової функції. Вибрані особини відображені в таблиці 2.24.

Таблиця 2.24 - Гени батьків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Батько 1	-4	14	14	11	0	8	13	18	-12	5	67	272
Батько 2	10	19	22	11	1	10	13	19	-13	-2	90	715

2. Дискретна рекомбінація

Проведення дискретної рекомбінації з використанням формули описаної на першій ітерації. Для першого нащадка значення коефіцієнту $\alpha = 0,4$. Для другого $\alpha = 0,3$. Знайдені гени нащадків відображені в таблиці 2.25.

Таблиця 2.25 - Гени нащадків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	2	16	17	11	0	9	13	18	-12	2	76	430
Нащадок 2	0	16	16	11	0	9	13	18	-12	3	74	388

3. Мутація

Проведення мутації нащадків з використанням формули описаної на першій ітерації. Для даної задачі ($m = 20$) значення коефіцієнтів відображені в таблиці 2.26. Гени нащадків після мутації відображені в таблиці 2.27.

Таблиця 2.26 - Значення коефіцієнтів для мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
$\pm$	+	+	+	-	-	-	+	+	-	++
	+	-	-	+	+	-	-	-	-	+

α	12	3,5	6	5,5	12,5	3,5	0	6	7	11
δ	0,29	0,32	0,50	0,22	0,36	0,47	0,21	0,69	0,29	0,63
	0,23	0,60	0,17	0,41	0,51	0,32	0,37	0,75	0,42	0,55

Таблиця 2.27 - Гени нащадків після мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Нащадок 1	5	17	20	10	-4	7	13	23	-14	9
Нащадок 2	3	13	15	13	7	7	13	14	-15	9

4. Реанімація

Проведення реанімації для генів нащадків, що мають недопустимі значення змінних. Недопустимі значення позначені червоним кольором в таблиці 2.27.

Перший нащадок має 2 недопустимі змінні, перша перевищує допустимий максимум, друга применшує допустимий мінімум. Другий нащадок має 3 недопустимі змінні. Перші 2 змінні перевищують допустимий максимум, третя применшує допустимий мінімум.

Для отримання допустимості змінним, що перевищують максимум варто присвоїти максимально допустиме значення, а змінним, що применшують мінімум, присвоїти мінімально допустиме значення.

Отримані значення відображенні зеленим кольором в таблиці 2.28.

Таблиця 2.28 - Гени нащадків після реанімації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	5	17	20	10	-4	7	13	19	-13	9	83	383
Нащадок 2	3	13	15	11	1	7	13	14	-13	9	74	279

5. Локальна оптимізація

Логіка оптимізації була описана на першій ітерації. Результати виконання на цій ітерації зображені на таблицях 2.29 та 2.30.

Таблиця 2.29 - Гени під час локальної оптимізації першого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	5	17	20	10	-4	7	13	19	-13	-2	72	528
2	5	17	20	10	1	7	13	19	-13	-2	77	564
3	10	17	20	10	1	7	13	19	-13	-2	82	649
4	10	17	20	10	1	10	13	19	-13	-2	85	652
5	10	17	20	11	1	10	13	19	-13	-2	86	677
6	10	19	20	11	1	10	13	19	-13	-2	88	695
7	10	19	22	11	1	10	13	19	-13	-2	90	715

Таблиця 2.30 - Гени під час локальної оптимізації другого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	3	13	15	11	1	7	13	14	-13	-2	63	421
2	10	13	15	11	1	7	13	14	-13	-2	70	516
3	10	13	15	11	1	10	13	14	-13	-2	73	518
4	10	19	15	11	1	10	13	14	-13	-2	78	574
5	10	19	15	11	1	10	13	19	-13	-2	83	642
6	10	19	22	11	1	10	13	19	-13	-2	90	715

6. Оновлення популяції

Заміна особин, що мають найменше значення цільової функції на створених нащадків (виділені жовтим кольором у таблиці 2.31).

Таблиця 2.31 - Популяція оновлених особин

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
-10	12	10	6	-1	5	13	11	0	12	58	78
7	16	14	7	-8	9	13	7	-8	10	67	149
10	19	22	11	1	10	13	19	-13	-2	90	715
-4	14	14	11	0	8	13	18	-12	5	67	272
9	12	18	7	-10	6	13	18	-3	6	76	390
-5	18	19	7	0	7	13	10	-10	6	65	113
1	12	15	4	-17	4	13	17	0	12	61	225
-1	19	17	10	-16	6	13	16	-9	11	66	164
10	19	22	11	1	10	13	19	-13	-2	90	715
10	19	22	11	1	10	13	19	-13	-2	90	715
10	19	22	11	1	10	13	19	-13	-2	90	715
-12	16	17	6	-2	6	13	18	-2	5	65	122
-6	18	11	4	-2	8	13	13	0	3	62	89
0	13	17	11	-23	10	13	16	-9	5	53	120
4	13	10	11	0	4	13	7	-4	3	61	241

7. Визначення рекорду

На другій ітерації значення рекорду буде тим самим оскільки воно дорівнює максимальному значення цільової функції серед усіх особин популяції.

Значення рекорду:

$$Record = 715.$$

$$Розв'язок: Record = 715.$$

Знаходження мінімуму цільової функції.

1. Генерація популяції із 15-ти особин.

Поділ особин на 2 групи випадковим чином. Згенеровані значення зображені в таблиці 2.32.

Таблиця 2.32 - Початкова популяція

Група	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	-10	12	10	6	-1	5	13	11	0	12	58	78
1	7	16	14	7	-8	9	13	7	-8	10	67	149
0	-11	18	13	8	-8	5	13	11	-10	14	53	-131
0	-4	14	14	11	0	8	13	18	-12	5	67	272
0	9	12	18	7	-10	6	13	18	-3	6	76	390
1	-5	18	19	7	0	7	13	10	-10	6	65	113
0	1	12	15	4	-17	4	13	17	0	12	61	225
1	-1	19	17	10	-16	6	13	16	-9	11	66	164
1	4	15	21	1	-1	7	13	11	-13	15	73	22
0	-6	16	17	6	-20	3	13	8	0	3	40	50
0	1	12	19	3	-16	10	13	9	-6	19	64	-88
0	-12	16	17	6	-2	6	13	18	-2	5	65	122
1	-6	18	11	4	-2	8	13	13	0	3	62	89
1	0	13	17	11	-23	10	13	16	-9	5	53	120
1	4	13	10	11	0	4	13	7	-4	3	61	241

Ітерація 1

2. Турнірний відбір

Вибір по одній особині з кожної групи, що мають найбільше значення цільової функції (виділено фіолетовим кольором) з табл. 2.32. Вибрані особини відображені в таблиці 2.33.

Таблиця 2.33 - Гени батьків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Батько 1	-11	18	13	8	-8	5	13	11	-10	14	53	-131
Батько 2	4	15	21	1	-1	7	13	11	-13	15	73	22

3. Дискретна рекомбінація

Використовується формула, що була описана на першій ітерації задачі на максимум. Для першого нащадка значення коефіцієнту $\alpha = 0,3$. Для другого $\alpha = 0,6$. Знайдені гени нащадків відображені в таблиці 2.34.

Таблиця 2.34 - Гени нащадків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Нащадок 1	-7	17	15	6	-6	6	13	11	-11	14
Нащадок 2	-2	16	18	4	-4	6	13	11	-12	15

4. Мутація

Для даної задачі ($m = 20$) значення коефіцієнтів відображені в таблиці 2.35.

Гени нащадків після мутації відображені в таблиці 2.36.

Таблиця 2.35 - Значення коефіцієнтів для мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
$\pm$	-	-	-	+	+	+	+	+	-	-
	+	-	-	+	+	-	-	-	+	+
α	12	3,5	6	5,5	12,5	3,5	0	6	7	11
δ	0,45	0,73	0,60	0,35	0,42	0,20	0,51	0,84	0,68	0,73
	0,66	0,68	0,43	0,83	0,31	0,55	0,70	0,21	0,27	0,22

Таблиця 2.36 - Гени нащадків після мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	-12	15	12	8	-1	6	13	16	-16	6	47	22
Нащадок 2	6	14	15	8	0	4	13	10	-10	17	78	166

5. Реанімація

Недопустимі значення позначені червоним кольором в таблиці 2.19. Перший нащадок має одну недопустиму змінну, яка применшує допустимий мінімум. Для отримання допустимості змінній варто присвоїти мінімально допустиме значення. Значення нових змінних відображені в таблиці 2.37.

Таблиця 2.37 - Гени нащадків після реанімації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	-12	15	12	8	-1	6	13	16	-13	6	50	45
Нащадок 2	6	14	15	8	0	4	13	10	-10	17	78	166

6. Локальна оптимізація

Пріоритетність на мінімізацію елементів залишається за змінними з максимальним значенням. Виняток становлять змінні x_2 , x_6 , x_{10} , оскільки вони мають від'ємну пару. В такому випадку вони навпаки максимізуються. Результати оптимізації відображені в таблицях 2.38 та 2.39

Таблиця 2.38 - Гени під час локальної оптимізації першого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	-12	15	12	8	-1	6	13	7	-13	6	41	-77
2	-12	15	10	8	-1	6	13	7	-13	6	39	-93
3	-12	15	10	0	-1	6	13	7	-13	6	31	-173
4	-12	15	10	0	-24	6	13	7	-13	6	8	-311
5	-12	15	10	0	-24	10	13	7	-13	6	12	-407
6	-12	15	10	0	-24	10	13	7	-13	20	26	-589
7	-14	15	10	0	-24	10	13	7	-13	20	24	-619
8	-14	19	10	0	-24	10	13	7	-13	20	28	-675

Таблиця 2.39 - Гени під час локальної оптимізації другого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	6	14	10	8	0	4	13	10	-10	17	72	124
2	6	14	10	8	0	4	13	7	-10	17	69	85
3	6	14	10	0	0	4	13	7	-10	17	61	5
4	-14	14	10	0	0	4	13	7	-10	17	41	-275
5	-14	14	10	0	-24	4	13	7	-10	17	17	-371
6	-14	14	10	0	-24	10	13	7	-10	17	23	-515
7	-14	14	10	0	-24	10	13	7	-13	17	20	-566
8	-14	19	10	0	-24	10	13	7	-13	17	25	-636
9	-14	19	10	0	-24	10	13	7	-13	20	28	-675

7. Оновлення популяції

Заміна особин, що мають найбільше значення цільової функції на створених нащадків (виділено жовтим кольором у табл. 2.40). Поділ особин на 2 групи випадковим чином.

Таблиця 2.40 - Популяція оновлених особин

Група	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	-10	12	10	6	-1	5	13	11	0	12	58	78
1	7	16	14	7	-8	9	13	7	-8	10	67	149
1	-11	18	13	8	-8	5	13	11	-10	14	53	-131
0	-14	19	10	0	-24	10	13	7	-13	20	28	-675
0	-14	19	10	0	-24	10	13	7	-13	20	28	-675
0	-5	18	19	7	0	7	13	10	-10	6	65	113
0	1	12	15	4	-17	4	13	17	0	12	61	225
1	-1	19	17	10	-16	6	13	16	-9	11	66	164
1	4	15	21	1	-1	7	13	11	-13	15	73	22
1	-6	16	17	6	-20	3	13	8	0	3	40	50
1	1	12	19	3	-16	10	13	9	-6	19	64	-88
0	-12	16	17	6	-2	6	13	18	-2	5	65	122
0	-6	18	11	4	-2	8	13	13	0	3	62	89
0	0	13	17	11	-23	10	13	16	-9	5	53	120
1	4	13	10	11	0	4	13	7	-4	3	61	241

8. Визначення рекорду

На першій ітерації рекордом буде мінімальне значення цільової функції серед усіх особин популяції, значення змінної:

$Record = -675.$

Ітерація 2

1. Турнірний відбір

Вибір по одній особині з кожної групи, що мають найбільше значення цільової функції. Вибрані особини відображені в таблиці 2.41.

Таблиця 2.41 - Гени батьків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Батько 1	-11	18	13	8	-8	5	13	11	-10	14	53	-131
Батько 2	-14	19	10	0	-24	10	13	7	-13	20	28	-675

2. Дискретна рекомбінація

Використовується формула, що була описана на першій ітерації задачі на максимум. Для першого нащадка значення коефіцієнту $\alpha = -0,2$. Для другого $\alpha=0,4$. Знайдені гени нащадків відображені в таблиці 2.42.

Таблиця 2.42 - Гени нащадків

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Нащадок 1	-10	18	14	10	-5	4	13	12	-9	13
Нащадок 2	-12	18	12	5	-14	7	13	9	-11	16

3. Мутація

Для даної задачі ($m = 20$) значення коефіцієнтів відображені в табл. 2.43. Гени нащадків після мутації відображені в таблиці 2.44.

Таблиця 2.43 - Значення коефіцієнтів для мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
$\pm$	+	+	+	+	-	-	-	-	+	+
	+	-	-	-	+	+	+	-	-	-
α	12	3,5	6	5,5	12,5	3,5	0	6	7	11
δ	0,95	0,66	0,06	0,09	0,42	0,20	0,72	0,05	0,93	0,93
	0,80	0,77	0,96	0,51	0,04	0,87	0,04	0,40	0,12	0,50

Таблиця 2.44 - Гени нащадків після мутації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Нащадок 1	1	20	14	10	-10	3	13	12	-3	23
Нащадок 2	-3	16	6	2	-14	10	13	7	-12	11

4. Реанімація

Недопустимі значення позначені червоним кольором в таблиці 2.44. Перший нащадок має 2 недопустимі змінні, які перевищують допустимий максимум. Другий нащадок має 1 змінну, яка применшує допустимий мінімум. Для отримання допустимості для першого нащадка недопустимим змінним варто присвоїти максимально допустиме значення.

Для отримання допустимості для другого нащадка недопустимій змінній варто присвоїти мінімально допустиме значення значення.

Значення нових змінних відображені в таблиці 2.45.

Таблиця 2.45 - Гени нащадків після реанімації

	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
Нащадок 1	1	19	14	10	-10	3	13	12	-3	20	79	225
Нащадок 2	-3	16	10	2	-14	10	13	7	-12	11	40	-209

5. Локальна оптимізація

Пріоритетність на мінімізацію елементів залишається за змінними з максимальним значенням. Виняток становить змінна x_6 , оскільки вона має від'ємну пару. В такому випадку вона навпаки максимізується. Результати оптимізації відображені в таблицях 2.45 та 2.46.

Таблиця 2.46 - Гени під час локальної оптимізації першого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	1	19	10	10	-10	3	13	12	-3	20	75	185
2	1	19	10	10	-10	3	13	7	-3	20	70	120
3	1	19	10	0	-10	3	13	7	-3	20	60	20
4	-14	19	10	0	-10	3	13	7	-3	20	45	-265
5	-14	19	10	0	-10	10	13	7	-3	20	52	-335
6	-14	19	10	0	-10	10	13	7	-13	20	42	-535
7	-14	19	10	0	-24	10	13	7	-13	20	28	-675

Таблиця 2.47 - Гени під час локальної оптимізації другого нащадка

Ітерації	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
1	-3	16	10	0	-14	10	13	7	-12	11	38	-229
2	-14	16	10	0	-14	10	13	7	-12	11	27	-405
3	-14	16	10	0	-14	10	13	7	-12	20	36	-513
4	-14	16	10	0	-14	10	13	7	-13	20	35	-533
5	-14	16	10	0	-24	10	13	7	-13	20	25	-633
6	-14	19	10	0	-24	10	13	7	-13	20	28	-675

6. Оновлення популяції

Заміна особин, що мають найбільше значення цільової функції на створених нащадків (виділені жовтим кольором у таблиці 2.48).

Таблиця 2.48 - Популяція оновлених особин

x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Сума	ЦФ
-10	12	10	6	-1	5	13	11	0	12	58	78
7	16	14	7	-8	9	13	7	-8	10	67	149
-11	18	13	8	-8	5	13	11	-10	14	53	-131

-14	19	10	0	-24	10	13	7	-13	20	28	-675
-14	19	10	0	-24	10	13	7	-13	20	28	-675
-5	18	19	7	0	7	13	10	-10	6	65	113
-14	19	10	0	-24	10	13	7	-13	20	28	-675
-1	19	17	10	-16	6	13	16	-9	11	66	164
4	15	21	1	-1	7	13	11	-13	15	73	22
-6	16	17	6	-20	3	13	8	0	3	40	50
1	12	19	3	-16	10	13	9	-6	19	64	-88
-12	16	17	6	-2	6	13	18	-2	5	65	122
-6	18	11	4	-2	8	13	13	0	3	62	89
0	13	17	11	-23	10	13	16	-9	5	53	120
-14	19	10	0	-24	10	13	7	-13	20	28	-675

7. Визначення рекорду

На другій ітерації значення рекорду буде тим самим, оскільки воно дорівнює мінімальному значенню цільової функції серед усіх особин популяції.

Значення рекорду:

$$Record = -675.$$

$$Розв'язок: Record = -675.$$

2.6 Джерела

1. ГУЛЯНИЦЬКИЙ Л. Ф. ПРИКЛАДНІ МЕТОДИ КОМБІНАТОРНОЇ ОПТИМІЗАЦІЇ. Навчальний посібник / Л. Ф. ГУЛЯНИЦЬКИЙ, О. Ю. МУЛЕСА., 2016. [1]-[2], [5]-[6]
2. Марусенко Р. В. АЛГОРИТМИ РОЙОВОГО ІНТЕЛЕКТУ ТА ЇХ ЗАСТОСУВАННЯ [Електронний ресурс] / Роман Вікторович Марусенко. – 2017. – Режим доступу до ресурсу:
<https://phm.cuspu.edu.ua/nauka/konferentsii/fizyka-tekhnologii-navchannia/99-2017/komp-iuterni-nauky-ta-informatsiini-tekhnologii/1122-alhorytmy-royovoho-intelektu-ta-yikh-zastosuvannya.html>. [3]
3. Гольдштейн А.Л. Оптимизация в среде MATLAB: учеб. пособие / А.Л. Гольдштейн. – Пермь: Изд-во Перм. нац. исслед. политехн. ун-та, 2015. – 192 с. [4]
4. Панченко Т. В. Генетические алгоритмы / Т. В. Панченко., 2007. – 88 с. [7]-[10]

3. ПЛАНУВАННЯ ЕКСПЕРИМЕНТІВ

3.1 Класифікація індивідуальних задач

Математична модель:

$$x_1x_2 + x_3x_4 + \dots x_{n-1}x_n \rightarrow extremum$$

$$\sum_{j=1}^n x_j \leq B,$$

$$a_j \leq x_j \leq b_j, \quad j = 1, 2, \dots, n,$$

$$x_j \in Z, \quad j = 1, 2, \dots, n,$$

$$a_j \leq 0, \quad j = 1, 2, \dots, n,$$

$$b_j > 0, \quad j = 1, 2, \dots, n,$$

$$B < \sum_{j=1}^n b_j.$$

Параметри ІЗ:

- n - кількість відрізків (парне число);
- B - константа;
- a_j - ліва границя x_j , $j = 1, 2, \dots, n$;
- b_j - права границя x_j , $j = 1, 2, \dots, n$.

ІЗ можна класифікувати за наступними ознаками:

- розмірність n ;
- значення константи B .

3.2 Параметри алгоритмів

Жадібний алгоритм:

- загальна кількість ітерацій роботи алгоритму (рівна розмірності задачі n).

Бджолиний алгоритм:

- загальна кількість ітерацій роботи алгоритму;
- кількість бджіл-розвідників (рівна розмірності задачі n);
- кількість бджіл-фуражирів (рівна розмірності задачі n);
- кількість ділянок для збору нектару (рівна розмірності задачі n).

Генетичний алгоритм:

- способи вибору батьків (аутбридінг, випадковий вибір обох батьків, вибір найбільш пристосованого і випадкового батьків, турнірний відбір);
- способи схрещування (дискретна рекомбінація);
- способи реанімації;
- способи мутації.

3.3 План експериментів

Експеримент №1

Мета: дослідити, як розмірність ІЗ впливає на значення ЦФ.

План: запустити набір алгоритмів, з певною початковою розмірністю ІЗ та з кроком її збільшення на кожній ітерації.

1. **for** $i := 1$ **to** *кількість ітерацій експерименту*
2. Згенерувати додаткові значення для вектору a та b кількістю заданого кроку
3. **for** $j := 1$ **to** 10
4. Виконати жадібний алгоритм
5. Виконати бджолиний алгоритм
6. Виконати генетичний алгоритм
7. **endfor**
8. Знайти середнє арифметичне значення для часу роботи та значення ЦФ для кожного алгоритму
9. **endfor**

Експеримент №2

Мета: дослідити, як кількість ітерацій роботи алгоритмів впливає на значення ЦФ.

План: запустити набір алгоритмів, з певною початковою кількістю ітерацій роботи алгоритму та з кроком її збільшення на кожній ітерації експерименту.

1. **for** $i := 1$ **to** *кількість ітерацій експерименту*
2. Збільшити кількість ітерацій на значення кроку
3. **for** $j := 1$ **to** 10
4. Виконати бджолиний алгоритм

5. Виконати **генетичний алгоритм**
6. **endfor**
7. Знайти середнє арифметичне значення для часу роботи та значення ЦФ для кожного алгоритму
8. **endfor**

Експеримент №3

Мета: дослідити, як значення константи B впливає на значення ЦФ.

План: запустити набір алгоритмів, з певним значенням константи B та з кроком її збільшення на кожній ітерації експерименту.

1. **for** $i := 1$ **to** *кількість ітерацій експерименту*
2. Збільшити значенням константи B на значення кроку
3. **for** $j := 1$ **to** 10
4. Виконати **жадібний алгоритм**
5. Виконати **бджолиний алгоритм**
6. Виконати **генетичний алгоритм**
7. **endfor**
8. Знайти середнє арифметичне значення для часу роботи та значення ЦФ для кожного алгоритму
9. **endfor**

Експеримент №4

Мета: дослідити, як різний тип селекції генетичного алгоритму впливає на значення ЦФ.

План: запустити генетичний алгоритм для таких типів селекції: аутбридінг, з випадковим вибором обох батьків, з вибором найбільш пристосованого і випадкового, турнірний відбір.

1. **for** $i := 1$ **to** *кількість ітерацій експерименту*
2. **for** $j := 1$ **to** 10
3. Виконати **генетичний алгоритм** використовуючи *аутбридінг*
4. Виконати **генетичний алгоритм** використовуючи *випадковий вибір обох батьків*
5. Виконати **генетичний алгоритм** використовуючи *вибір найбільш пристосованого та випадкового батьків*
6. Виконати **генетичний алгоритм** використовуючи *турнірний відбір*
7. **endfor**

8. Знайти середнє арифметичне значення для часу роботи та значення ЦФ для алгоритму
9. **endfor**

Експеримент №5

Мета: дослідити, як наявність та відсутність локальної оптимізації генетичного алгоритму впливає на значення ЦФ.

План: запустити генетичний алгоритм із застосуванням локальної оптимізації та без.

1. **for** $i := 1$ **to** *кількість ітерацій експерименту*
2. **for** $j := 1$ **to** 10
3. Виконати **генетичний алгоритм** із локальною оптимізацією
4. Виконати **генетичний алгоритм** без локальної оптимізації
5. **endfor**
6. Знайти середнє арифметичне значення для часу роботи та значення ЦФ для алгоритму
7. **endfor**

4. ОПИС ПРОГРАМНОГО ПРОДУКТУ

4.1 Опис функціональної моделі

Програмний продукт (ПП) містить наступні складові:

а) робота з індивідуальною задачею (ІЗ):

- 1) введення вручну даних ІЗ;
- 2) генерація (випадковим чином) даних ІЗ;
- 3) збереження в файл/читання з файлу/редагування даних ІЗ;
- 4) розв'язання ІЗ усіма розробленими методами (з виведенням на екран відповідних розв'язків та значень ЦФ);

б) експериментальне дослідження розроблених алгоритмів:

- 1) задання діапазонів зміни параметрів експериментів:
 - розмірність задач (від; до; крок);
 - діапазоні зміни коефіцієнтів (ЦФ та обмежень).
- 2) генерація множини ІЗ;
- 3) розв'язання множини згенерованих ІЗ усіма розробленими методами;
- 4) виведення результатів експериментів:
 - порівняння часу роботи алгоритмів відповідно розмірності ІЗ;
 - порівняння значень ЦФ для різних типів селекції генетичного алгоритму відповідно до кількості ітерацій роботи алгоритму;
 - порівняння значень ЦФ для генетичного алгоритму з локальною оптимізацією та без відповідно до кількості ітерацій роботи алгоритму.

4.2 Рішення з інформаційного забезпечення

4.2.1 Вхідні дані

Вхідними даними для роботи з ІЗ є введені дані з клавіатури (рис. 4.2), випадково згенеровані дані з введенням з клавіатури значенням розмірності (рис. 4.1), зчитані дані з файлу (рис. 4.3). Параметри експериментів такі як кількість ітерацій та крок вводяться з клавіатури (рис. 4.4), дані ІЗ для експериментів зчитуються з файлу.

```
Choose input (1 - random IT; 2 - custom IT; 3 - saved IT; 4 - run experiments 0 - exit): 1
Enter the number of elements: 10
a = [ 30,  2, -14, 40, -36, -11, 28, -41, -6,  2];
b = [ 63, 32, 27, 70,  1, 81, 32, 50, 51, 60];
n = 10; B = 263;

Greedy algorithm results. Time: 0,0009206s
Maximum value = 3976. Iterations: 10
Minimum value = -1264. Iterations: 10

Bee algorithm results. Time: 0,0063764s
Maximum value = 8058. Iterations: 300

Genetic algorithm results. Time: 0,2771029s
Maximum value = 6292. Iterations: 300
Minimum value = -5518. Iterations: 300

Save input data? (1 - yes; 0 - no):
```

Рис. 4.1 – Генерація даних ІЗ

```

Choose input (1 - random IT; 2 - custom IT; 3 - saved IT; 4 - run experiments 0 - exit): 2
Enter the number of elements: 4
Enter values of a vector:
1: 6
2: 4
3: 7
4: 9

Enter values of b vector:
1: 8
2: 3
Value must be higher or equal then from a vector. Try again: 10
3: 15
4: 20

Enter value of B constant: 50
a = [ 6, 4, 7, 9];
b = [ 8, 10, 15, 20];
n = 4; B = 50;

Greedy algorithm results. Time: 3,2E-06s
Maximum value = 335. Iterations: 4
Minimum value = 87. Iterations: 4

Bee algorithm results. Time: 0,0022108s
Maximum value = 322. Iterations: 300

Genetic algorithm results. Time: 0,0359346s
Maximum value = 335. Iterations: 300
Minimum value = 87. Iterations: 300

Save input data? (1 - yes; 0 - no): 0
Press any key to continue...

```

Рис. 4.2 – Ввід даних ІЗ з клавіатури

```

Choose input (1 - random IT; 2 - custom IT; 3 - saved IT; 4 - run experiments 0 - exit): 3
a = [ 6, -16, 28, -46, -47, -5, -45, 13, -22, -30];
b = [ 8, 10, 43, 40, 93, 24, 72, 68, 22, 9];
n = 10; B = 182;

Greedy algorithm results. Time: 4,6E-06s
Maximum value = 1800. Iterations: 10
Minimum value = -1074. Iterations: 10

Bee algorithm results. Time: 0,0044942s
Maximum value = 8430. Iterations: 300

Genetic algorithm results. Time: 0,2828843s
Maximum value = 11483. Iterations: 300
Minimum value = -7866. Iterations: 300

Press any key to continue...

```

Рис. 4.3 – Зчитування даних ІЗ з файлу

```

Choose input (1 - random IT; 2 - custom IT; 3 - saved IT; 4 - run experiments 0 - exit): 4
Enter the number of algorithm iterations to start (possibly 10): 10
Enter the step of iterations: 10
Enter the number of experiment iterations (possibly 10): 10
Enter the step of of B constant (possibly 10): 5

```

Рис. 4.4 – Ввід параметрів експеременту

4.2.2 Вихідні дані

Результатом роботи програми для роботи з ІЗ є вивід в консолі інформації про результати роботи трьох розроблених алгоритмів у вигляді знайдених значень цільової функції, кількості ітерацій, та часу роботи для кожного алгоритму, є можливість запису ІЗ в файл. Результатом програми для роботи з експериментами є Excel документ зі звітом по результатах досліджень.

4.2.3 Опис інформаційного забезпечення

Для збереження ІЗ ПП використовує запис даних в json-файл. Результати експериментів записуються в Excel документ.

4.3 Рішення з програмного забезпечення

4.3.1 Опис обраних засобів розробки

ПП реалізовано у вигляді консольного застосунку на платформі .Net 5. Серед сторонніх бібліотек було використано EPPlus версії 5.6.4.

4.3.2 Архітектура програмного забезпечення

4.3.2.1 Схема архітектури ПП

ПП складається з одного проекту.

4.3.2.2 Діаграма класів (об'єктів)

Діаграма класів (об'єктів) наведена на рис. 4.1.

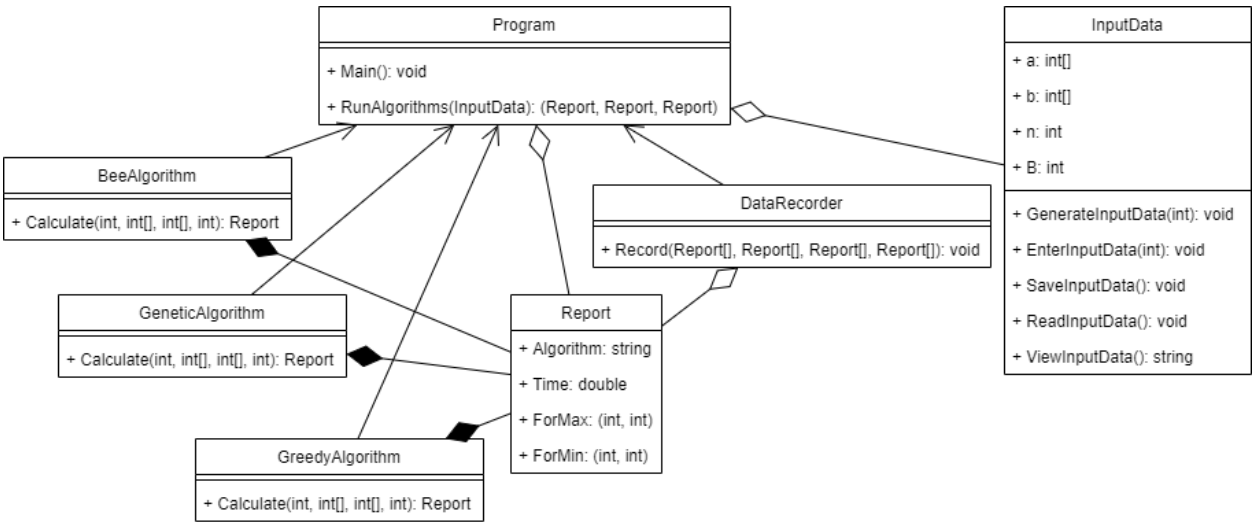


Рис. 4.5 - Діаграма класів (об'єктів)

4.3.2.3 Специфікація функцій

Специфікація функцій наведена у табл. 4.1.

Таблиця 4.1 – Опис основних функцій проекту

Назва	Вхідні параметри	Вихідні параметри	Опис
Calculate	Дані ІЗ	Звіт з результатами роботи алгоритму	Обраховує дані ІЗ згідно алгоритму і повертає знайдені дані у формі звіту
RunAlgorithms	Дані ІЗ	Звіти з результатами роботи алгоритмів	Запускає всі 3 алгоритми на обраховує час їх роботи
Record	Колекції звітів про результати роботи алгоритмів	—	Записує результати роботи всіх експериментів в Excel документ
GenerateInputData	Розмірність ІЗ	—	Генерує допустимі параметри ІЗ відповідно до розмірності

<i>EnterInputData</i>	Розмірність ІЗ	–	Дозволяє ввести параметри ІЗ з клавіатури
<i>SaveInputData</i>	–	–	Записує дані ІЗ у файл
<i>ReadInputData</i>	–	–	Зчитує дані ІЗ з файлу
<i>ViewInputData</i>	–	Рядок типу string з інформацією про поточну ІЗ	Повертає інформацією про поточну ІЗ

4.4 Керівництво користувача

4.4.1 Інструкція користувача

Користувач працює з ПП через консоль.

Для роботи з ІЗ він має можливість вибрати 3 види вводу вхідних даних:

- ввід з клавіатури;
- випадкова генерація вхідних;
- зчитування даних з файлу.

При вводі з клавіатури користувачу необхідно ввести розмірність задачі, масив значень вектору a , масив значень вектору b , значення константи B . При випадковій генерації користувачу необхідно ввести розмірність ІЗ з клавіатури. Після виконання алгоритмів та виведення їх відповідних результатів користувач має можливість записати ІЗ до файлу.

Для роботи з експериментами користувач має ввести з клавіатури початкове значення кількості ітерацій, крок ітерацій та кількість ітерацій самого експерименту на кількість ітерацій та крок для зміни значень константи B для експерименту на значення B .

Результати роботи усіх експериментів користувач має отримати в створеному Excel документі, що знаходиться поряд з виконуваним файлом.

4.4.2 Методика випробувань

Випробування працездатності розроблених алгоритмів відбулась на ІЗ з розмірністю від 2 до 30 з різними значеннями константи В. Всі отримані значення ЦФ були допустимими, тобто отримані значення при задачі на максимум були більші ніж значення відповідної задачі на мінімум, приріст константи В призвів до очікуваного збільшення значення ЦФ при задачі на максимум.

5. ДОСЛІДЖЕННЯ РОЗРОБЛЕНИХ АЛГОРИТМІВ

5.1 Дослідження впливу розмірності ІЗ на значення ЦФ

Для дослідження ми розглядаємо три випадки:

- задачу на максимум, де можуть використовуватися усі три розроблені алгоритми (жадібний, бджолиний та генетичний);
- задачу на мінімум, де застосовуються жадібний та генетичний алгоритми;
- та часова залежність трьох алгоритмів від обраного параметру.

Тут обраним параметром є розмірність задачі n . Для проведення експерименту була задана відповідна розмірність $n \in \{2, \dots, 50\}$. Оскільки за умовою задачі n – парне число, то крок збільшення буде мати значення 2.

На рис. 5.1 зображений графік залежності розмірності ІЗ від значення ЦФ для задачі на максимум. При порівнянні алгоритмів за точністю можна зазначити, що саме генетичний алгоритм знаходить найкраще значення ЦФ.

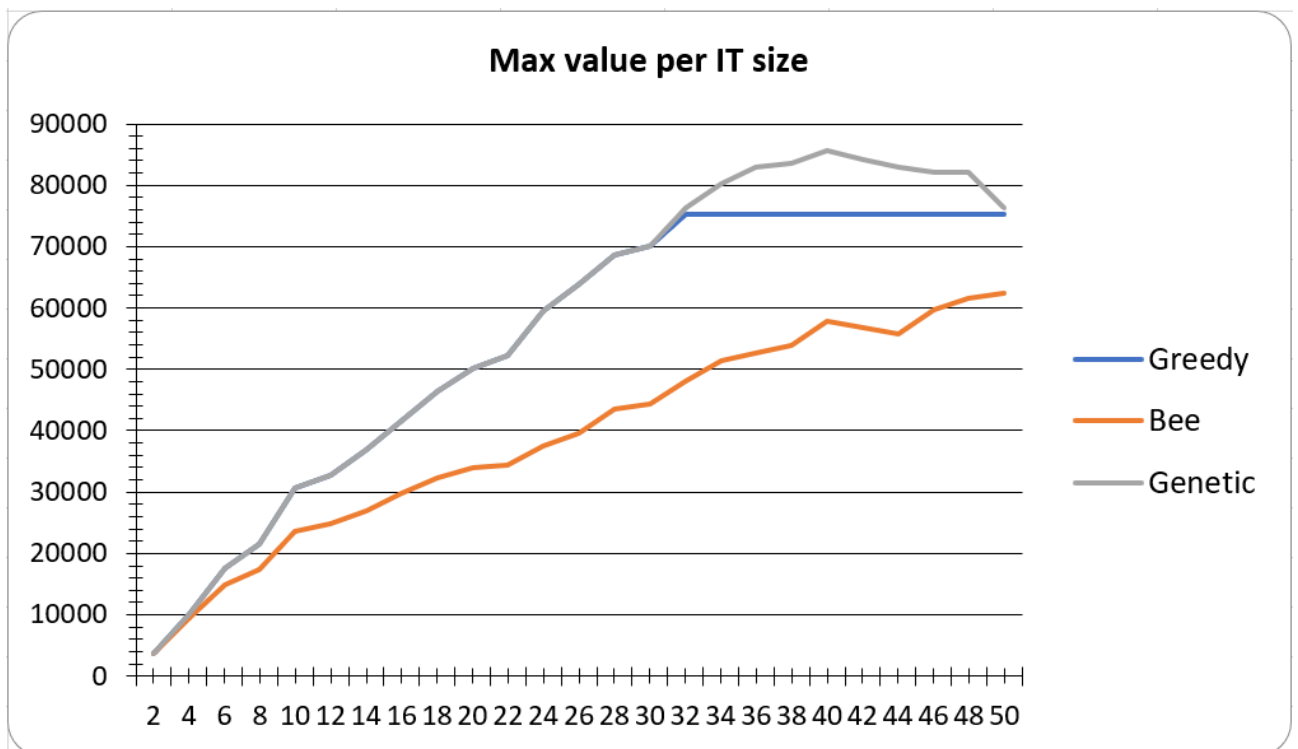


Рис. 5.1 – Графік залежності розмірності ІЗ від значення ЦФ (задача на максимум)

Результати порівняння алгоритмів за точністю при розв'язанні задачі на мінімум наведені на рис. 5.2. Крива генетичного алгоритму, що відображає залежність між розмірністю ІЗ та значенням ЦФ, є стрімкішою та направлена вниз. Це означає, що генетичний алгоритм знаходить кращі мінімальні значення ЦФ та не залишається у локальному мінімумі.

Можна стверджувати, що генетичний алгоритм є найбільш ефективним для вирішення задач на максимум та мінімум серед усіх інших алгоритмів, що були розглянуті у цій роботі.

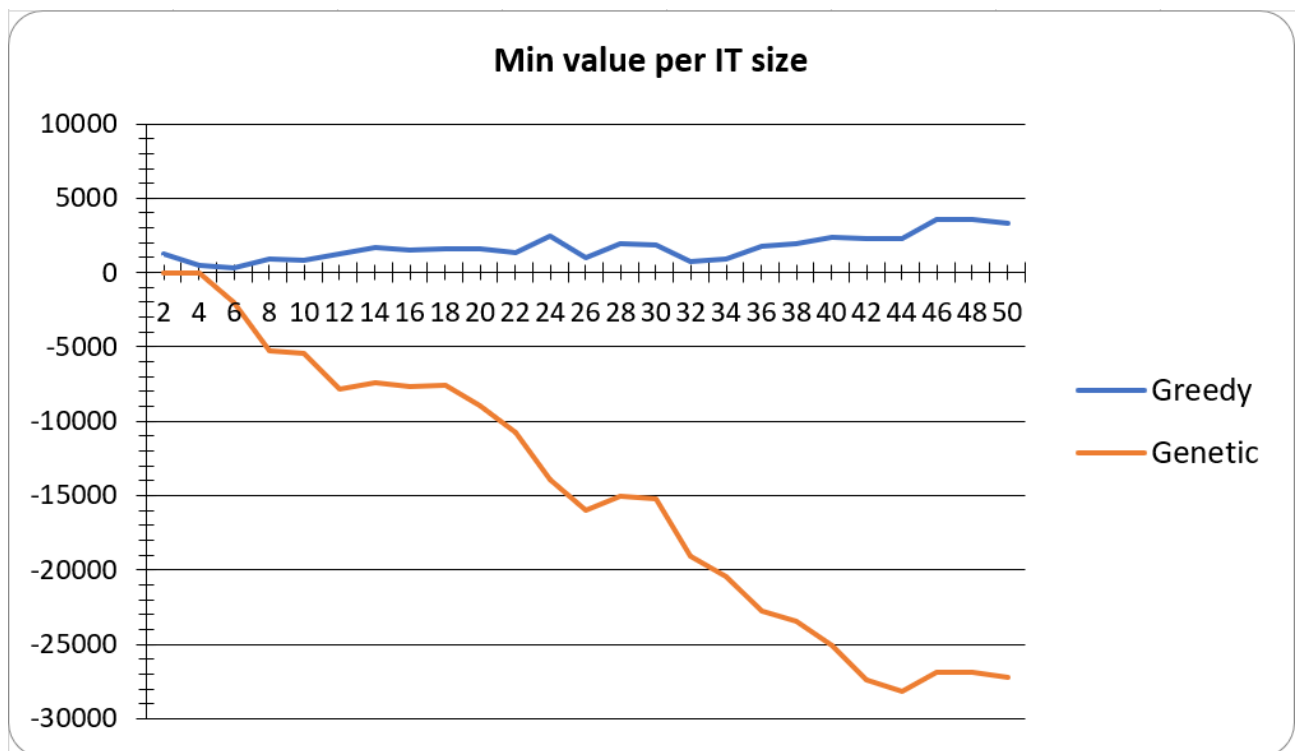


Рис. 5.2 – Графік залежності розмірності ІЗ від значення ЦФ (задача на мінімум)

На рисунку 5.3 можна побачити криві оцінок трудомісткості алгоритмів. Експериментально доведено, що алгоритми мають наступні оцінки:

- жадібний алгоритм – $O(n)$;
- бджолиний алгоритм – $O(n)$;
- генетичний алгоритм – $O(n^2)$.

Таким чином жадібний та бджолиний алгоритми мають лінійну залежність від розмірності ІЗ, а генетичний – квадратичну. Як і було зазначено генетичний

алгоритм має таку обчислювальну складність через наявність локальної оптимізації.

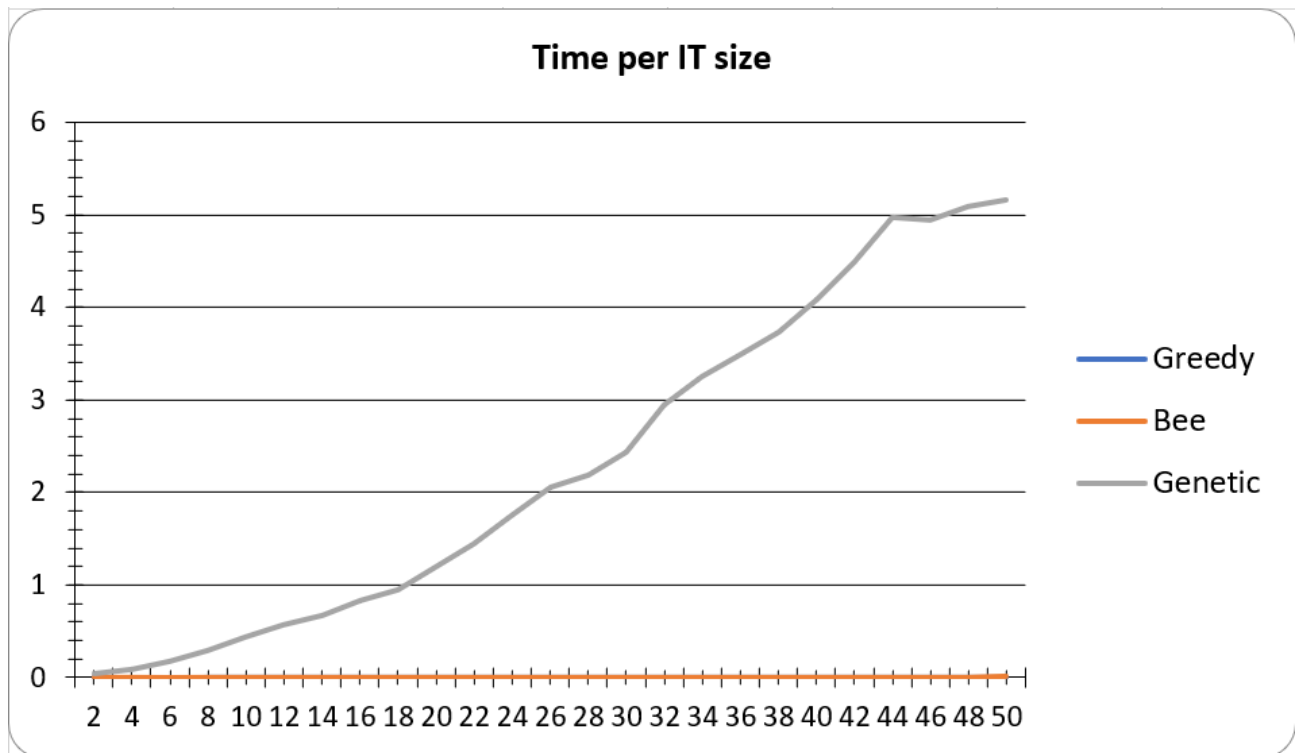


Рис. 5.3 – Графік залежності розмірності ІЗ від часу роботи алгоритму

5.2 Дослідження впливу кількості ітерацій роботи алгоритмів на значення ЦФ

Для дослідження залежності кількості виконаних ітерацій алгоритму та значенням ЦФ будуть розглядатися ті самі типи експерименту, що і у п. 5.1 (задача на максимум, задача на мінімум та час від обраного параметру).

У цьому експерименті було досліджено такий параметр, як кількість ітерацій. Максимальна кількість ітерацій роботи алгоритму належить проміжку $\{10, \dots, 1000\}$.

На рисунку 5.4 зображено, що генетичний алгоритм знаходить кращі результати за бджолиний за ту саму кількість виконаних ітерацій. Це також вказує на те, що хоча генетичний алгоритм і відноситься до наближених методів обчислення, але працює краще для задачі на максимум.

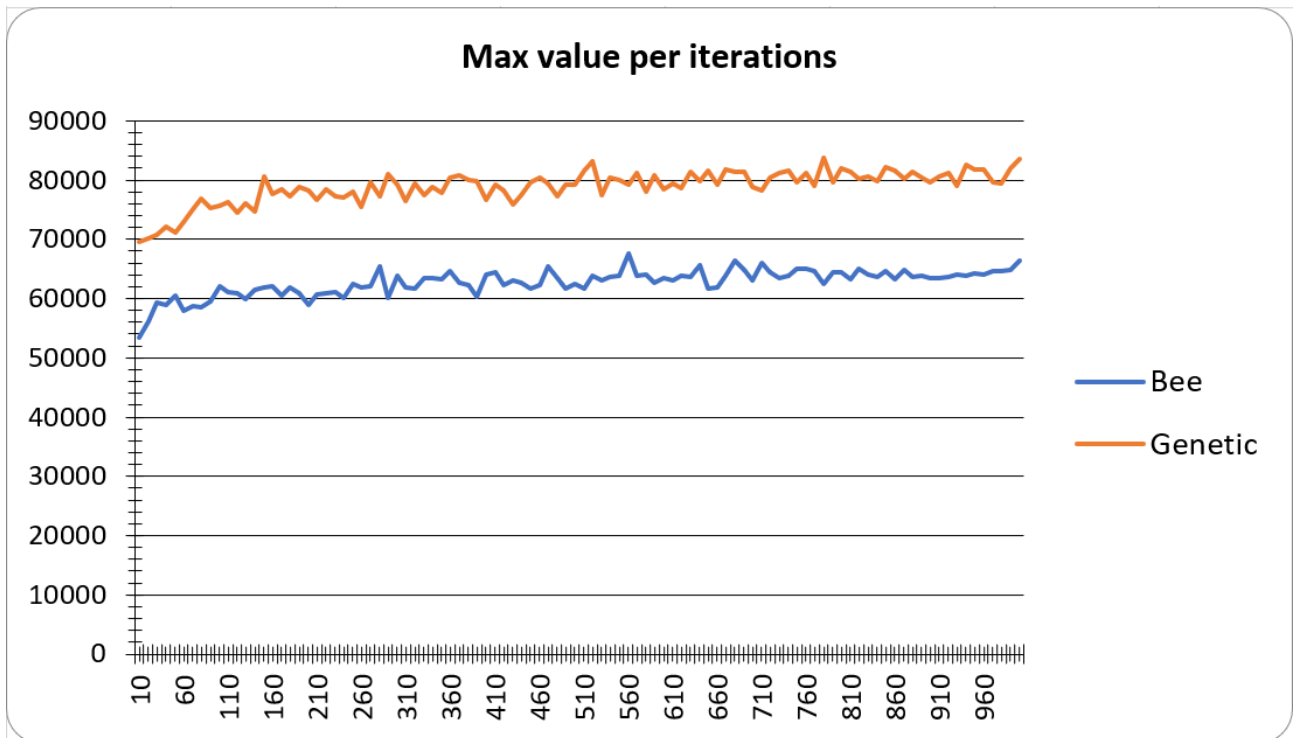


Рис. 5.4 – Графік залежності кількості ітерацій від значення ЦФ (задача на максимум)

Для порівняння ефективності роботи алгоритмів, які вирішують задачу на мінімум, був побудований графік на рис. 5.5. Оскільки жадібний алгоритм працює за n ітерацій, то недоцільно його порівнювати із генетичним.

Тоді генетичний алгоритм є найбільш ефективним для пошуку рішення задачі на мінімум та на максимум серед усіх, що були розглянуті у даній роботі.

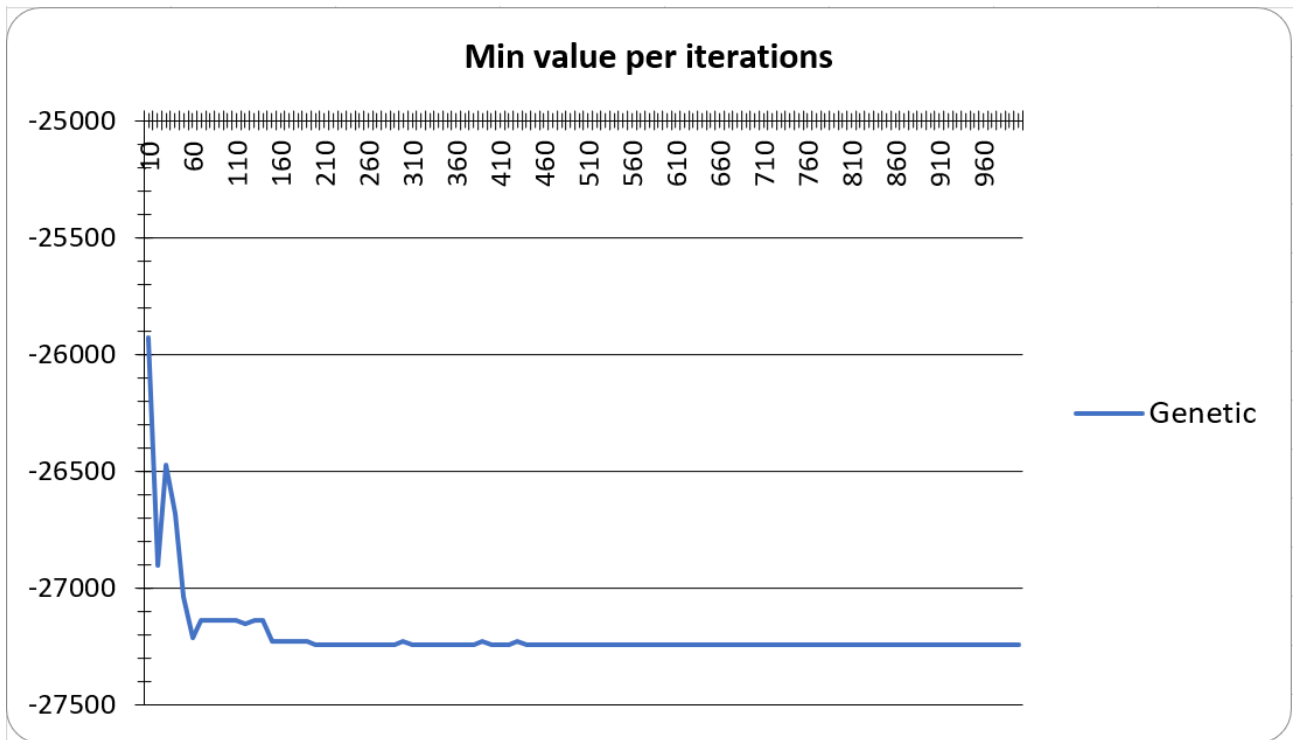


Рис. 5.5 – Графік залежності кількості ітерацій від значення ЦФ (задача на мінімум)

Третім графіком даного експерименту (рис. 5.6) було показано, що генетичний алгоритм працює довше бджолиного через його більшу обчислювальну складність, яка була показана на рис. 5.3.

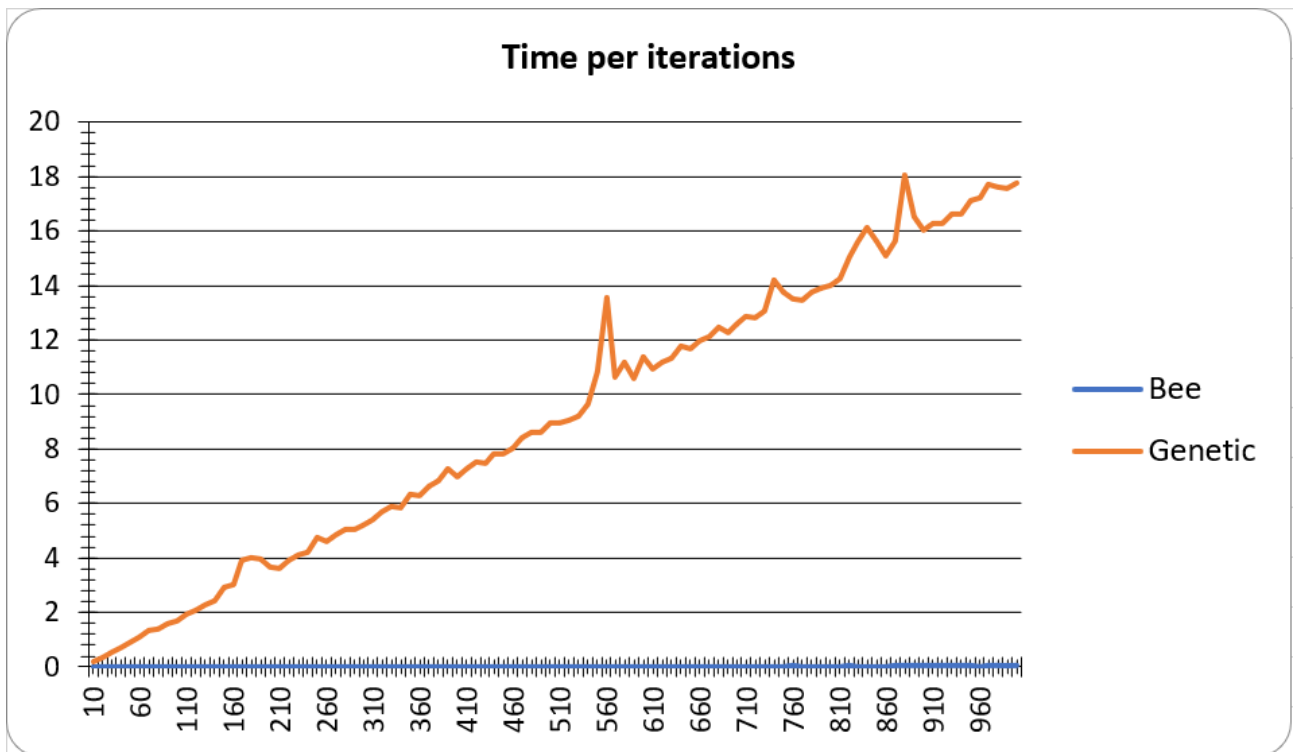


Рис. 5.6 – Графік залежності кількості ітерацій від часу роботи алгоритму

5.3 Дослідження впливу значення константи В на значення ЦФ

Дане дослідження покаже залежність константи В від значення ЦФ. Константа В вказує на максимальну суму елементів вектору x та суму добутку пар цього вектору при пошуку значення ЦФ.

На графіку залежності значення константи В від значення ЦФ (задача на максимум) (рис. 5.7) крива генетичного алгоритму має специфічні значення до $B=867$. Після цієї точки алгоритм працює ефективно. Цей розкид показує, що при малих значеннях константи В генетичний алгоритм може мати похибку обчислень. При великих значеннях цієї константи алгоритм працює ефективніше.

Варто зазначити, що значення ЦФ не змінюється при зміні величини константи В при вирішенні задачі бджолиним алгоритмом.

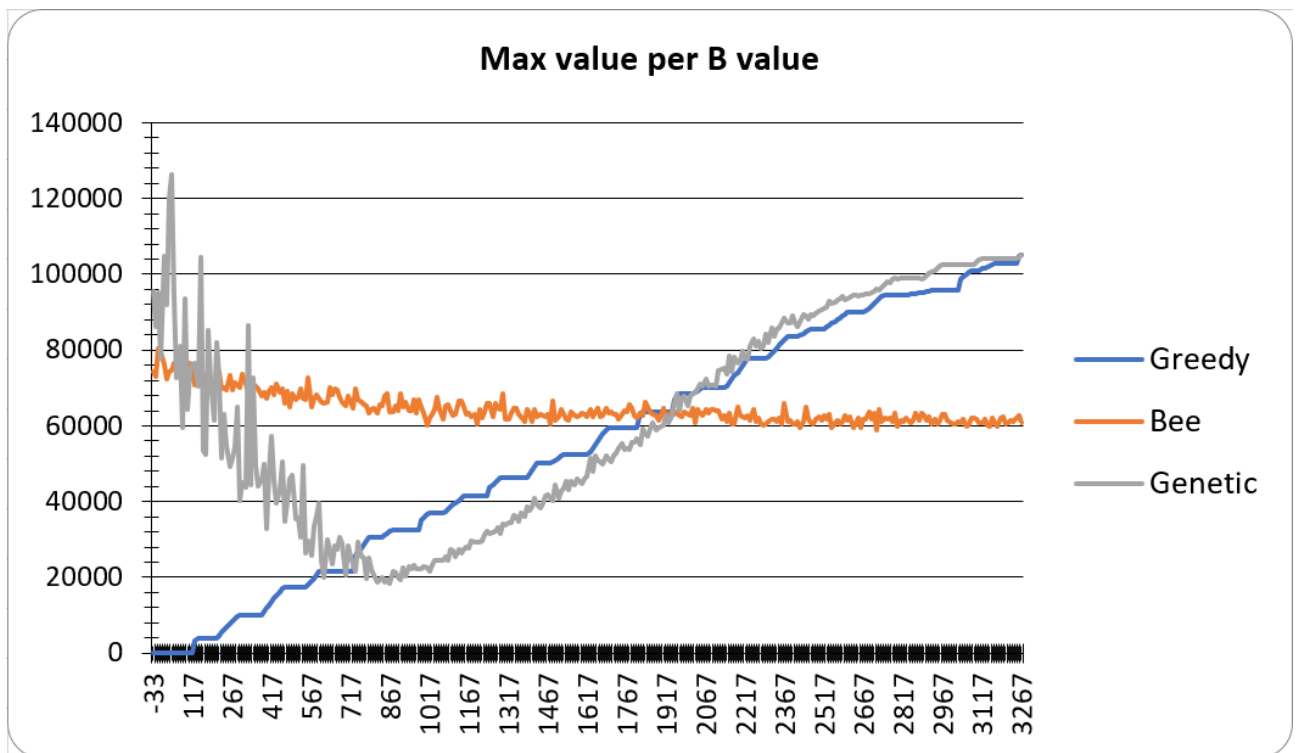


Рис. 5.7 – Графік залежності значення константи В від значення ЦФ (задача на максимум)

Такі самі результати генетичний алгоритм показує на рис. 5.8. Тут значення ЦФ при значенні константи $B=720$ та менше алгоритм може давати похибку обчислення. Після зазначеного значення константи В генетичний

алгоритм потрапляє у локальний оптимум та не змінює своє значення ЦФ через використання типу селекції турнірний відбір.

За великих значень B бджолиний алгоритм має меншу обчислювальну складність (рис. 5.9). Крива генетичного алгоритму має такий вигляд через такі фактори:

- до $B=667$ основна обчислювальна складність складається із процесу реанімації для генерування початкової популяції;
- після $B=667$ основна обчислювальна складність складається із процесу локальної оптимізації та реанімації.

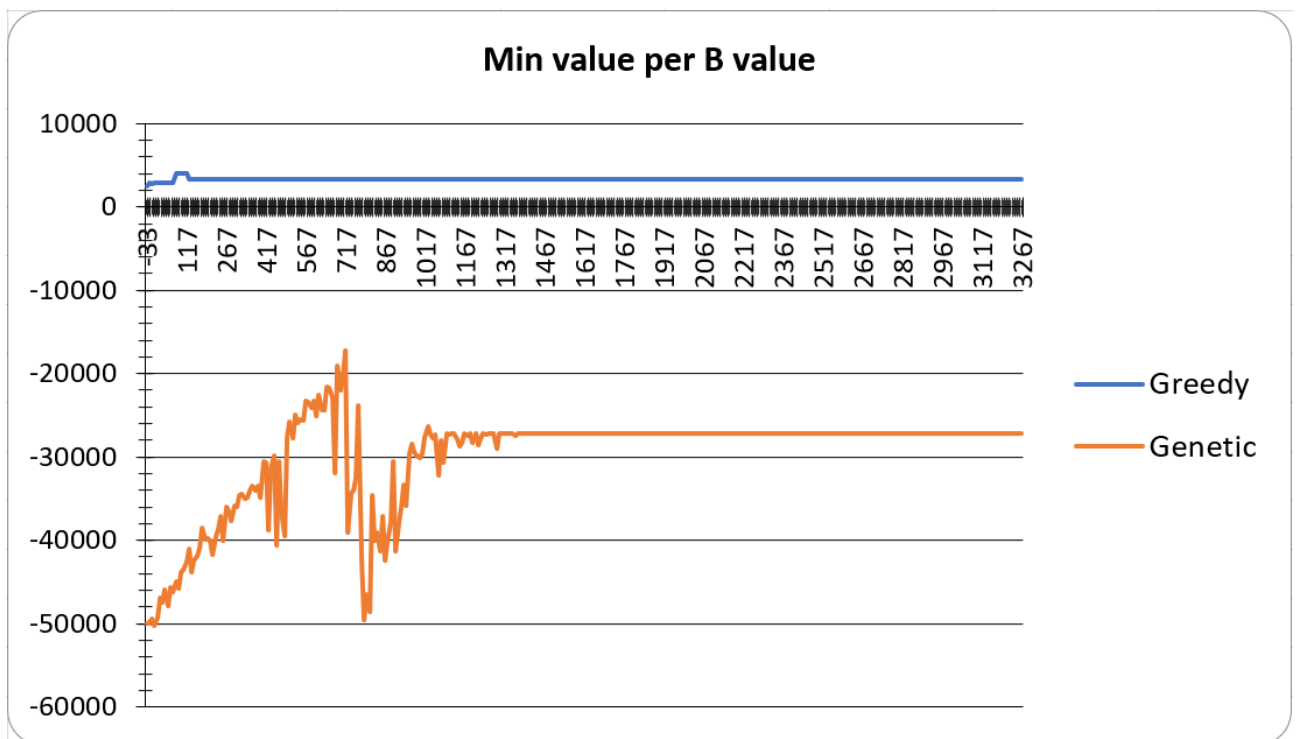


Рис. 5.8 – Графік залежності значення константи B від значення ЦФ (задача на мінімум)

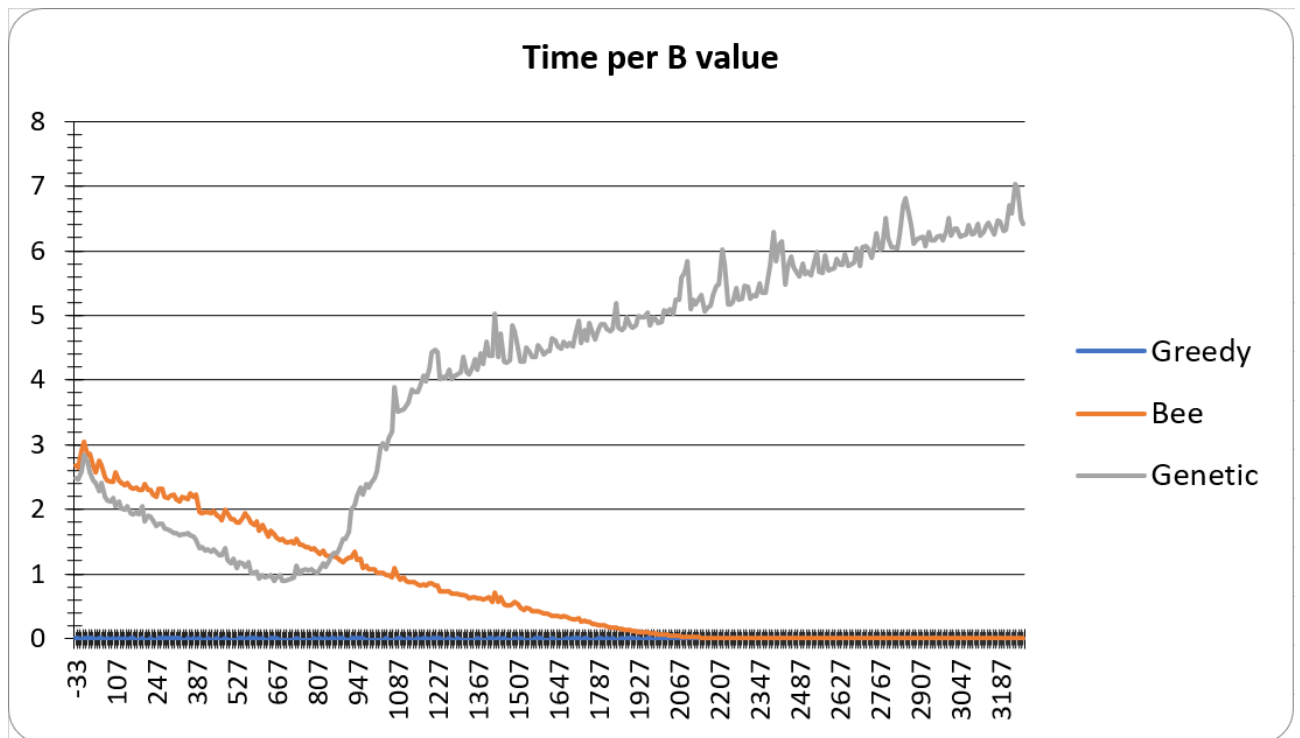


Рис. 5.9 – Графік залежності значення константи B від часу роботи алгоритму

5.4 Дослідження впливу типів селекції генетичного алгоритму на значення ЦФ

При виконанні даного експерименту були розглянуті наступні типи селекції (механізми вибору батьків із популяції для схрещування):

- аутбридінг;
- випадковий вибір обох батьків;
- вибір найбільш пристосованого і випадкового батьків;
- турнірний відбір.

Також при трактуванні змісту отриманих результатів було розглянуто задачі з такими вхідними розмірностями:

- $n=14$ (рисунки 5.11, 5.13 та 5.15);
- $n=50$ (рисунки 5.10, 5.12 та 5.14).

Це дозволило детальніше розглянути отримані результати.

Таким чином графіки залежності кількості ітерації від значення ЦФ при використанні різних типів селекції (задача на максимум) (рис. 5.10 та 5.11) дали такі результати:

- при використанні типу селекції турнірного відбору генетичний алгоритм входить в локальний максимум;
- тип селекції з випадковим вибором обох батьків та з вибором найбільш пристосованого і випадкового дають найкращі результати.

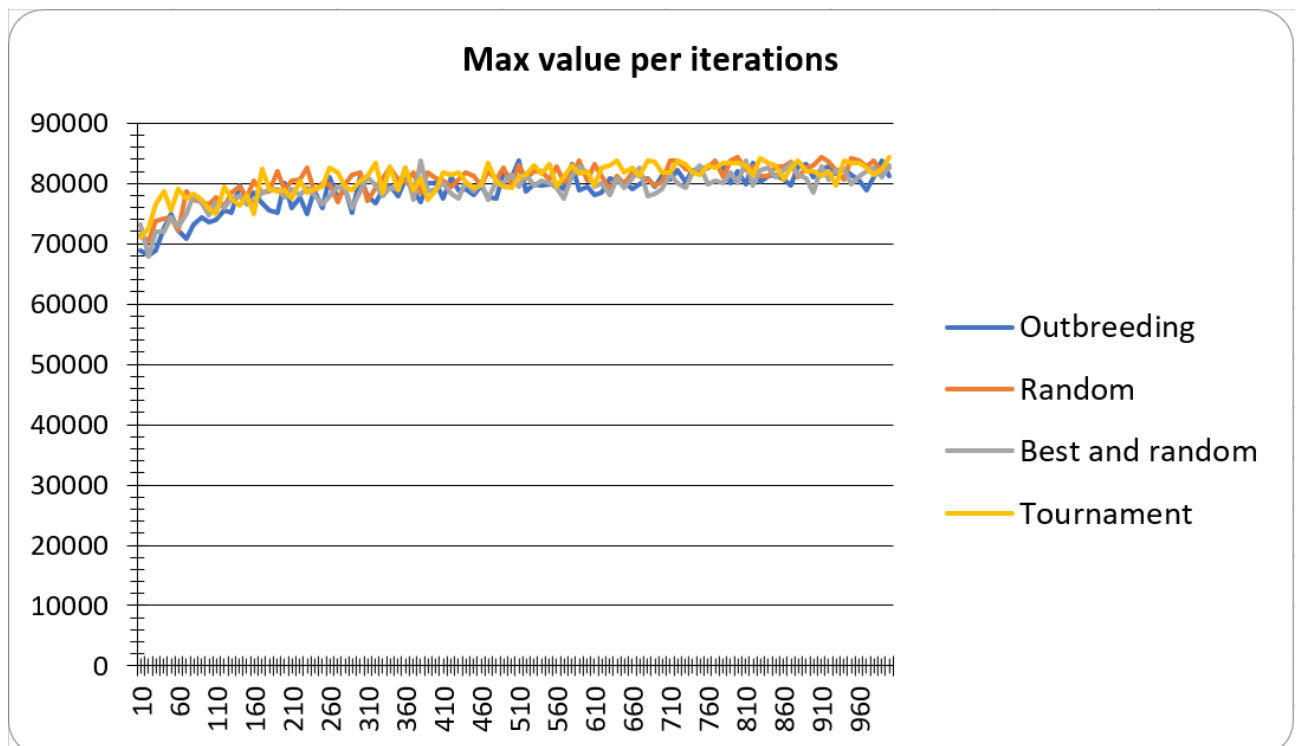


Рис. 5.10 – Графік залежності кількості ітерації від значення ЦФ при використанні різних типів селекції (задача на максимум)

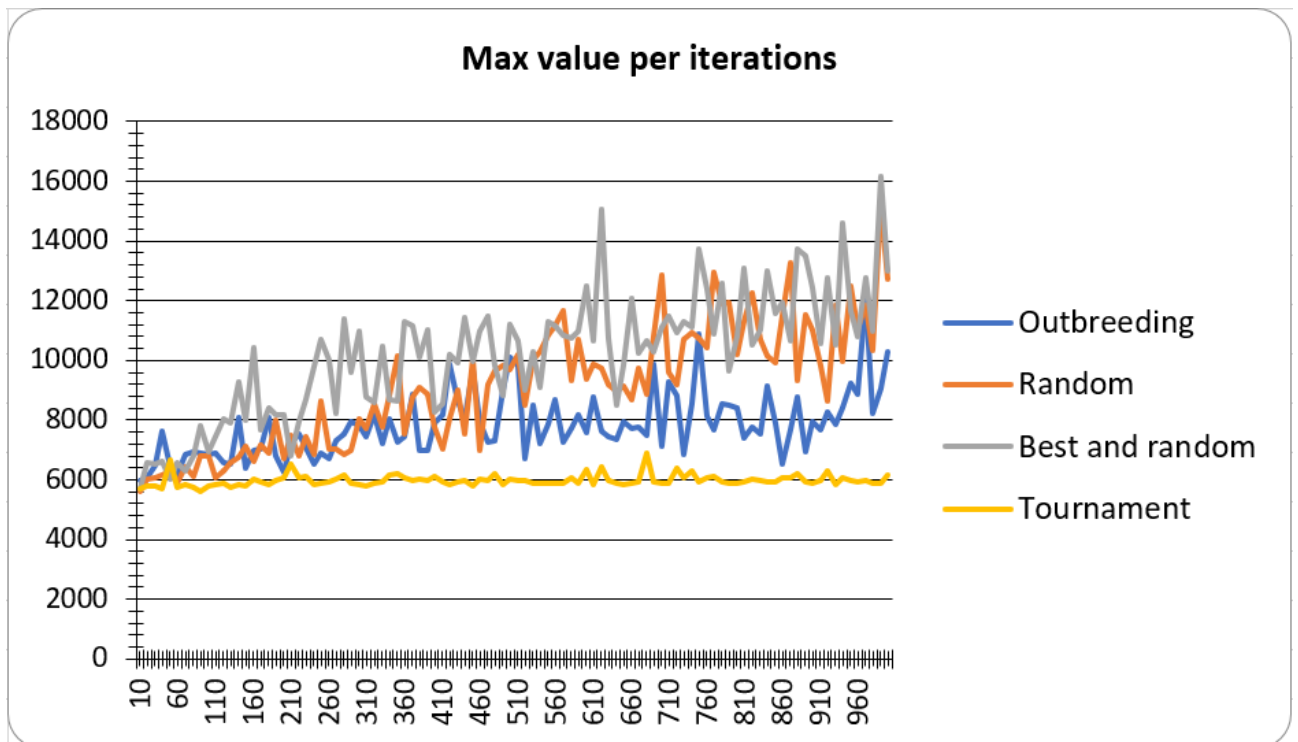


Рис. 5.11 – Графік залежності кількості ітерації від значення ЦФ при використанні різних типів селекції (задача на максимум)

Графіки на рисунках 5.12 та 5.13 побудовані для вирішення задачі на мінімум. Трактуювання отриманих результатів має той самий зміст, що і для задачі на максимум.

Таким чином найкращі типи селекції при використанні генетичного алгоритму є випадковий вибір обох батьків та вибір найбільш пристосованого і випадкового батьків.

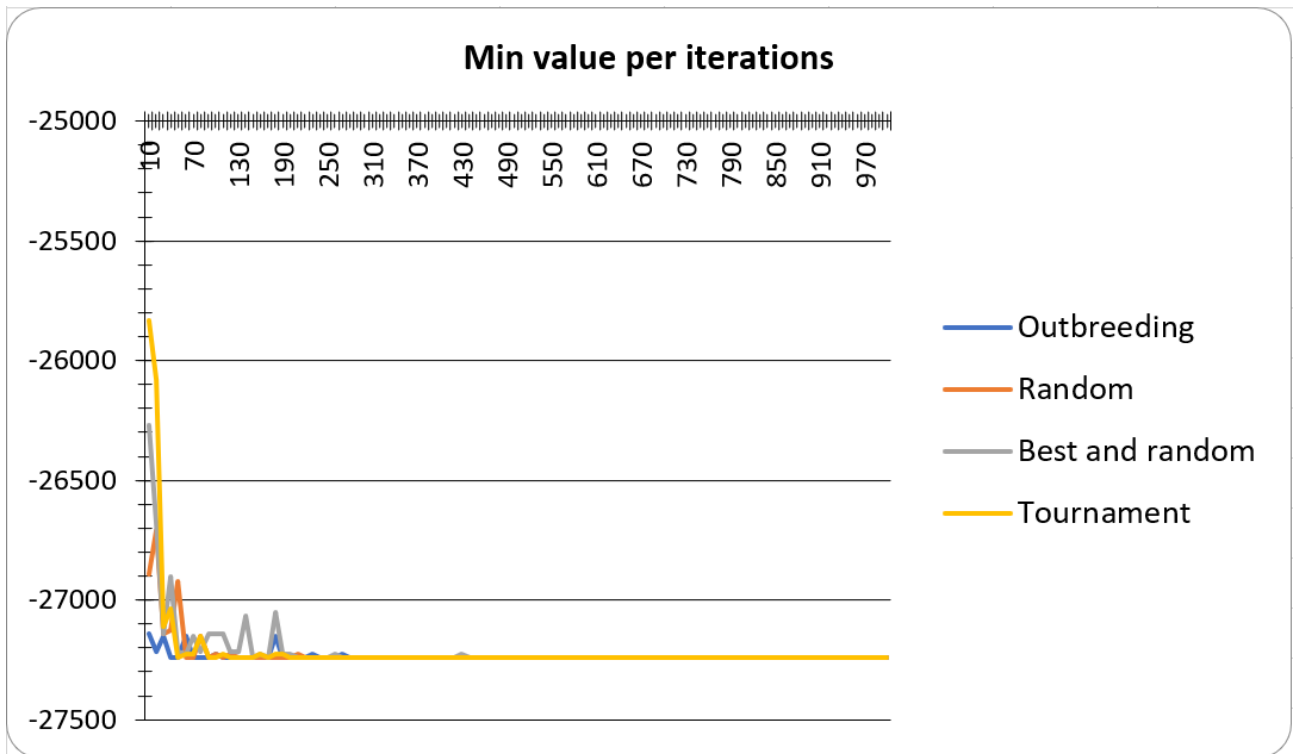


Рис. 5.12 – Графік залежності кількості ітерації від значення ЦФ при використанні різних типів селекції (задача на мінімум)

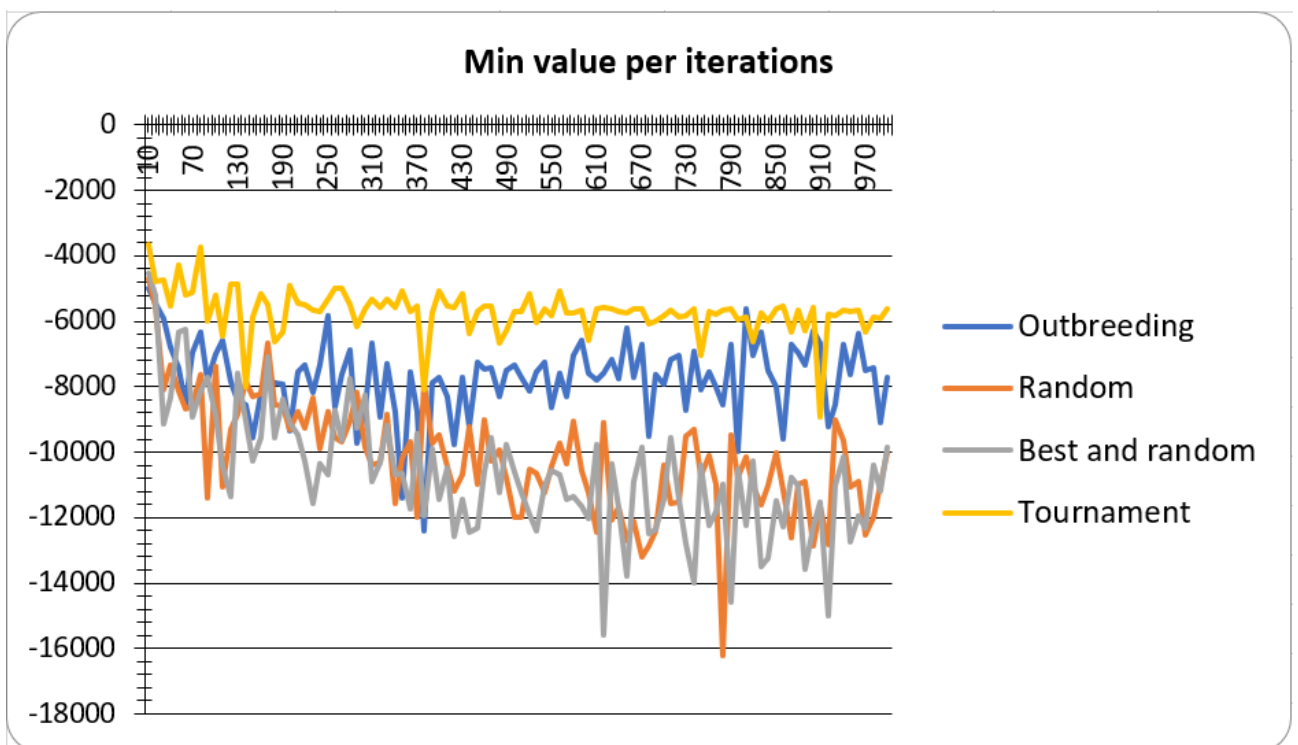


Рис. 5.13 – Графік залежності кількості ітерації від значення ЦФ при використанні різних типів селекції (задача на мінімум)

На графіках залежності кількості ітерації від часу роботи алгоритму при використанні різних типів селекції (рис. 5.14 та 5.15) можна побачити, що усі типи селекції займають асимптотично однаковий час виконання.

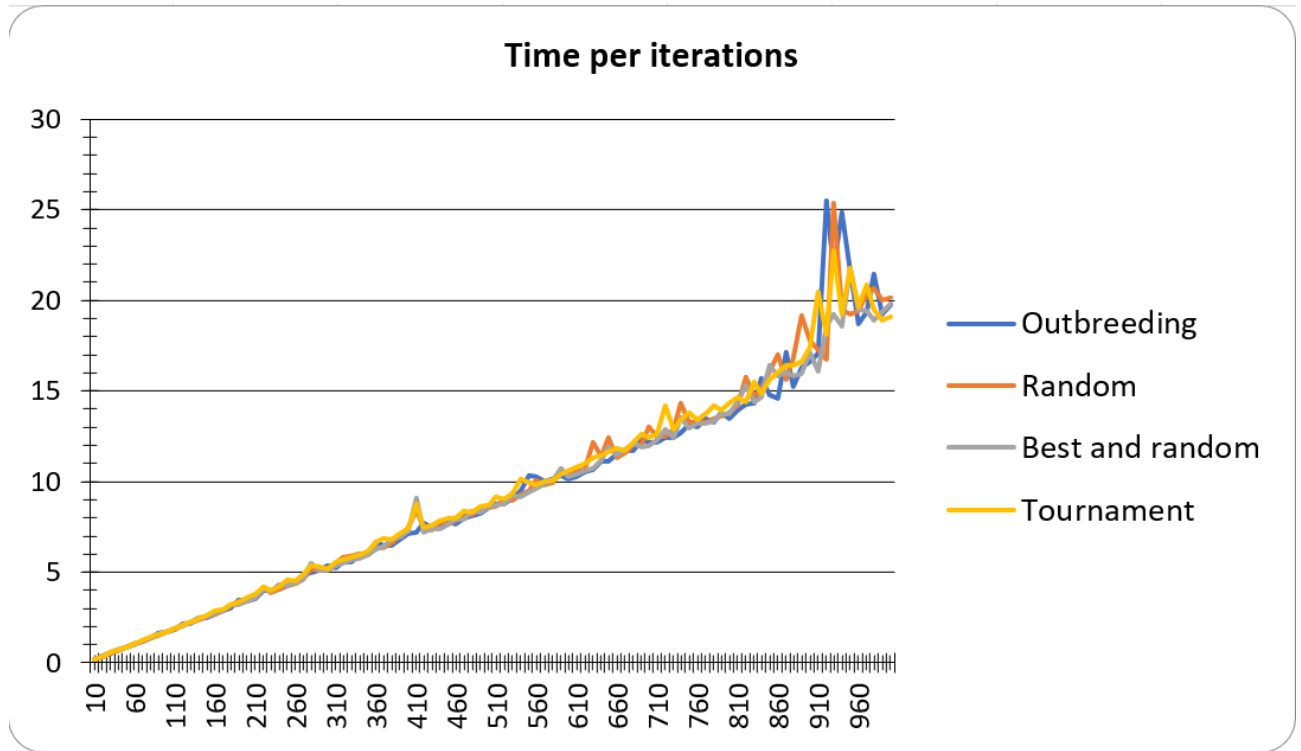


Рис. 5.14 – Графік залежності кількості ітерації від часу роботи алгоритму при використанні різних типів селекції

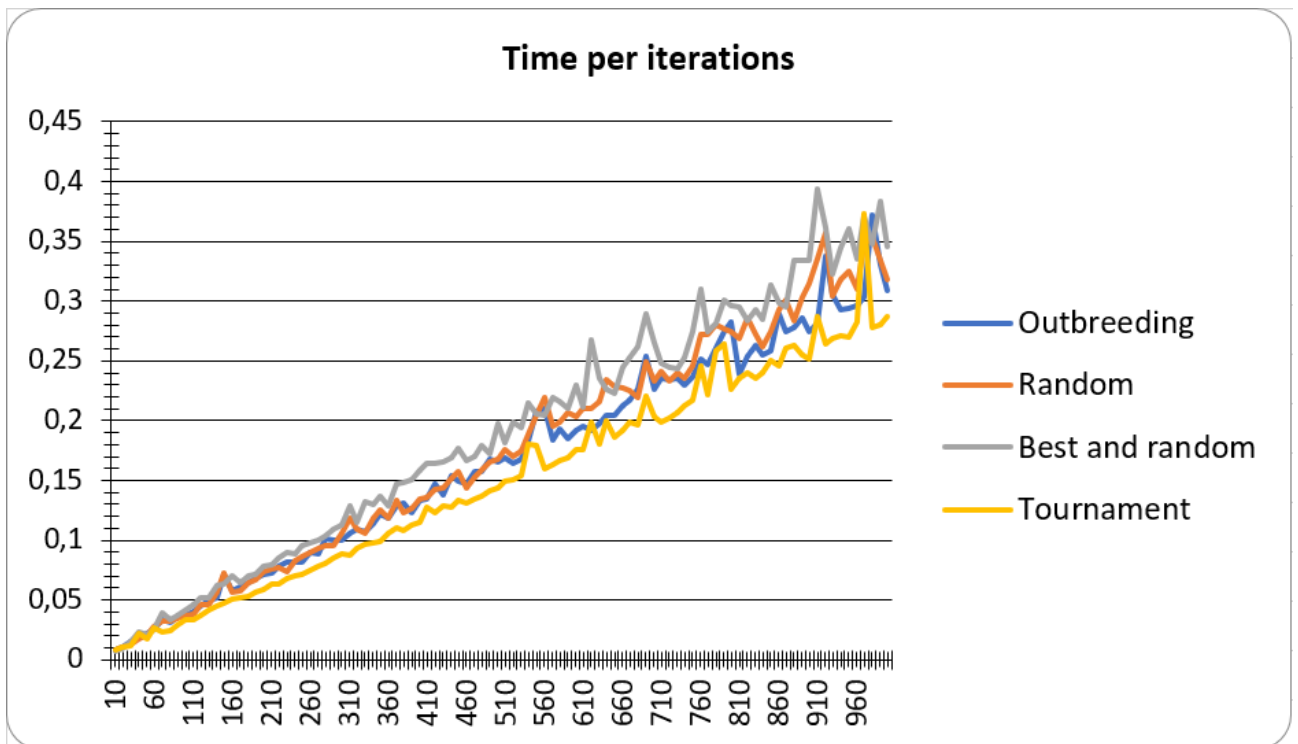


Рис. 5.15 – Графік залежності кількості ітерації від часу роботи алгоритму при використанні різних типів селекції

5.5 Дослідження впливу наявності та відсутності локальної оптимізації генетичного алгоритму на значення ЦФ.

В цьому дослідженні виконується генетичний алгоритм із застосуванням локальної оптимізації та без неї.

На графіках залежності кількості ітерації від значення ЦФ при застосуванні локальної оптимізації та її відсутності для ІЗ на максимум та мінімум (рис. 5.16 та 5.17) показано, що генетичний алгоритм із відсутньою локальною оптимізацією має відносно гірші результати ніж при застосуванні оптимізації.

Також варто зазначити, що оскільки при задачі на мінімум (рис. 5.17) при локальній оптимізації вектор-розв'язок x приймає мінімальне значення вектору a як його нижньої допустимої границі, то крива буде матиме вигляд лінії (лінійний зв'язок). Розв'язок при локальній оптимізації не покращується після 100-ї ітерації та набуває лінійного вигляду.

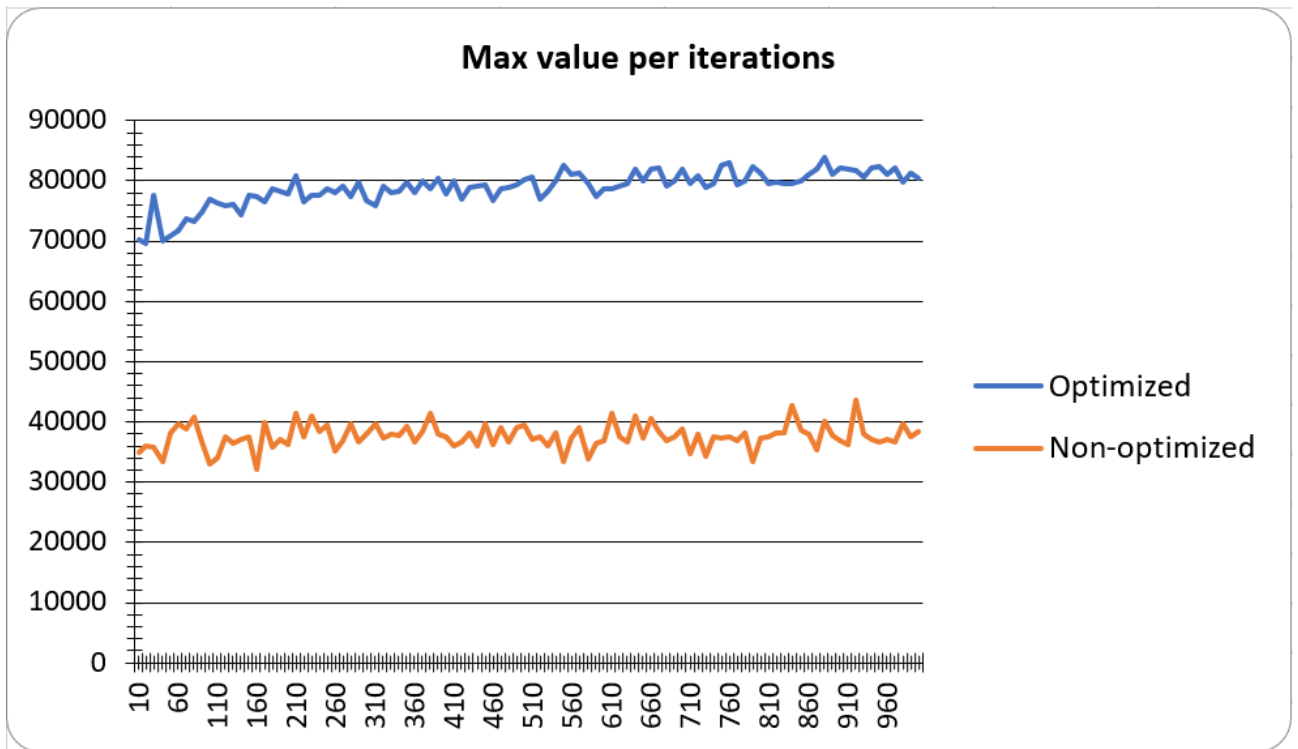


Рис. 5.16 – Графік залежності кількості ітерації від значення ЦФ при застосуванні локальної оптимізації та її відсутності (задача на максимум)

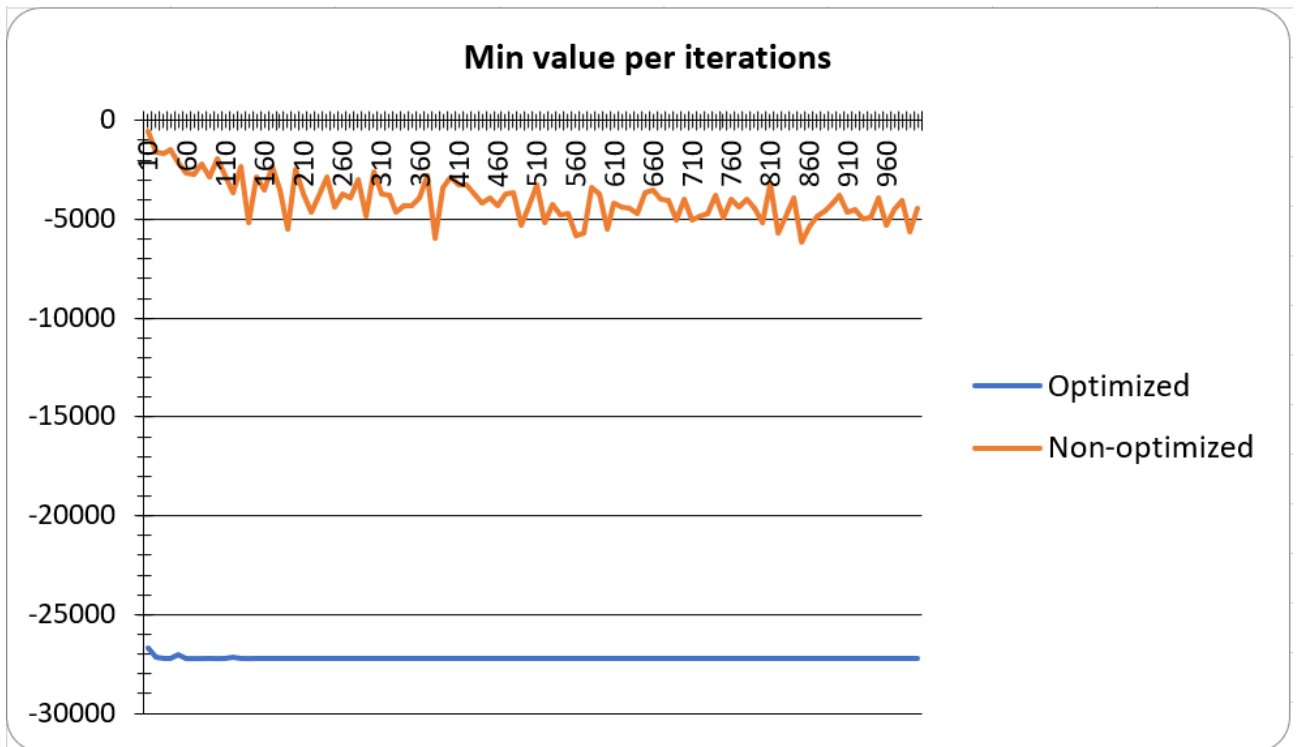


Рис. 5.17 – Графік залежності кількості ітерації від значення ЦФ при застосуванні локальної оптимізації та її відсутності (задача на мінімум)

На рис. 5.18 оптимізаційна крива стрімко прямує вгору через її додаткову обчислювальну складність у порівнянні із кривою, що відповідає за відсутність оптимізації у генетичному алгоритмі.

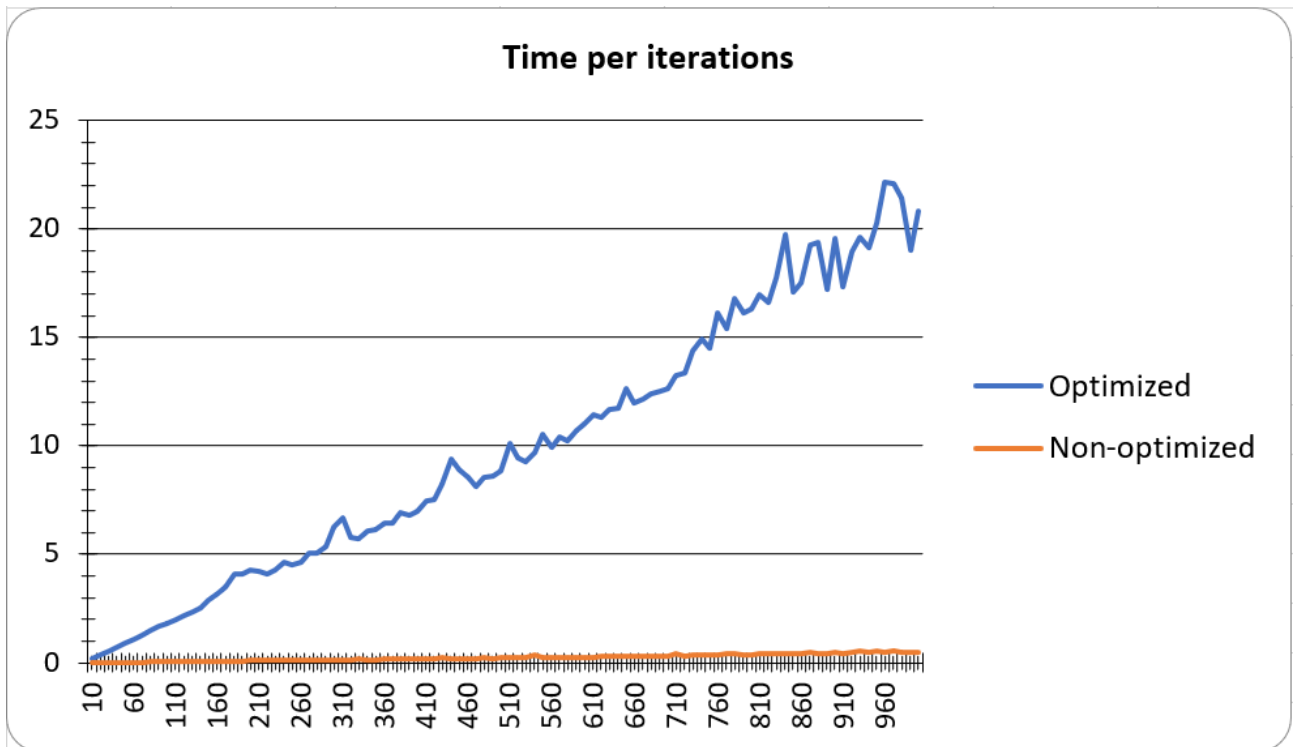


Рис. 5.18 – Графік залежності кількості ітерації від часу роботи алгоритму при застосуванні локальної оптимізації та її відсутності